

在 Linux 上部署 Docker 环境

本文也有配套视频，建议结合食用：(给个三连吧 www) <https://www.bilibili.com/video/BV1684y1z7L6>

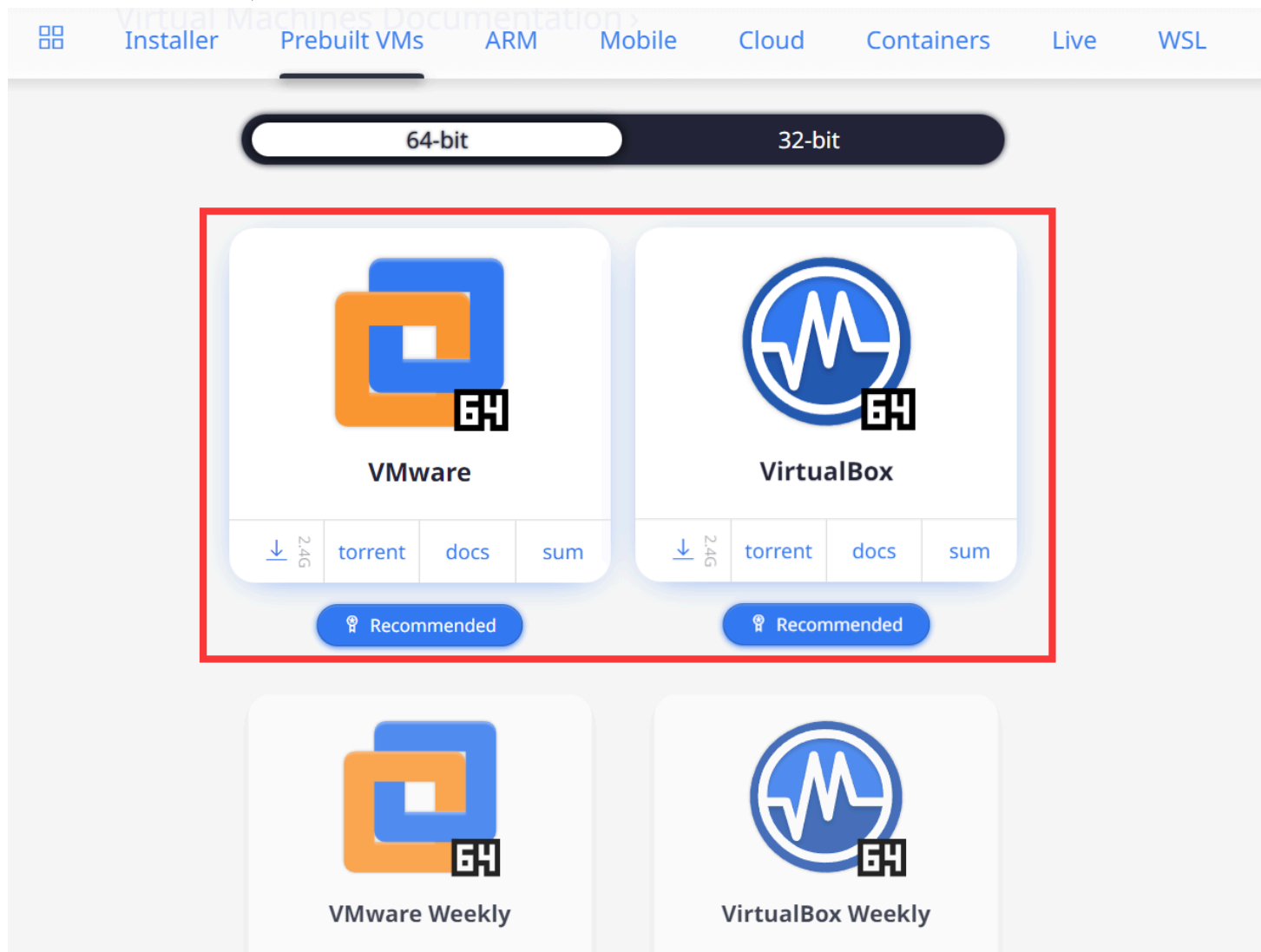
About

本文档完整的流程可以帮助您在 **Kali Linux** 上完成 Docker 环境，并且基于 VSCode 的 Remote 功能，利用 SSH 协议，构建 VSCode + SSH + Docker 的工作流，提高效率。同样，本文档也考虑了非 Kali，用于开发 / 生产环境的 **Ubuntu / Debian** 系统，如果您想要在开发 / 生产中使用该工作流，可以直接从 **Get Docker** 或者 **Build SSH + VSCode + Docker Workflow** 章节开始。笔者水平有限，错误疏漏之处在所难免，烦请各位师傅斧正。以及非常感谢 **陈橘墨 (@Randark_JMT)** 师傅提供的帮助。

Get Kali

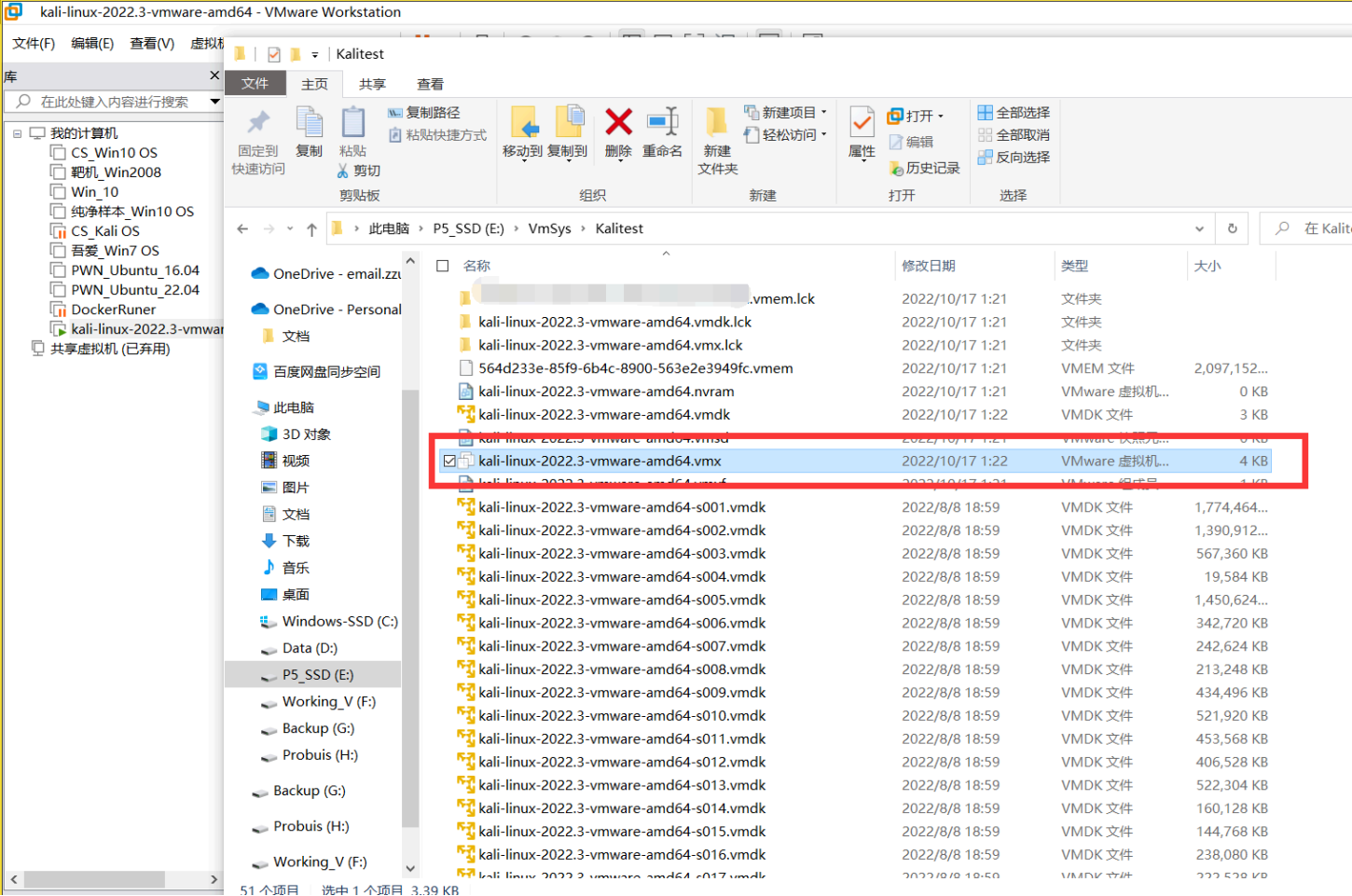
Download

在 Kali 官网下载即可，建议直接下载封装好的虚拟机版本：

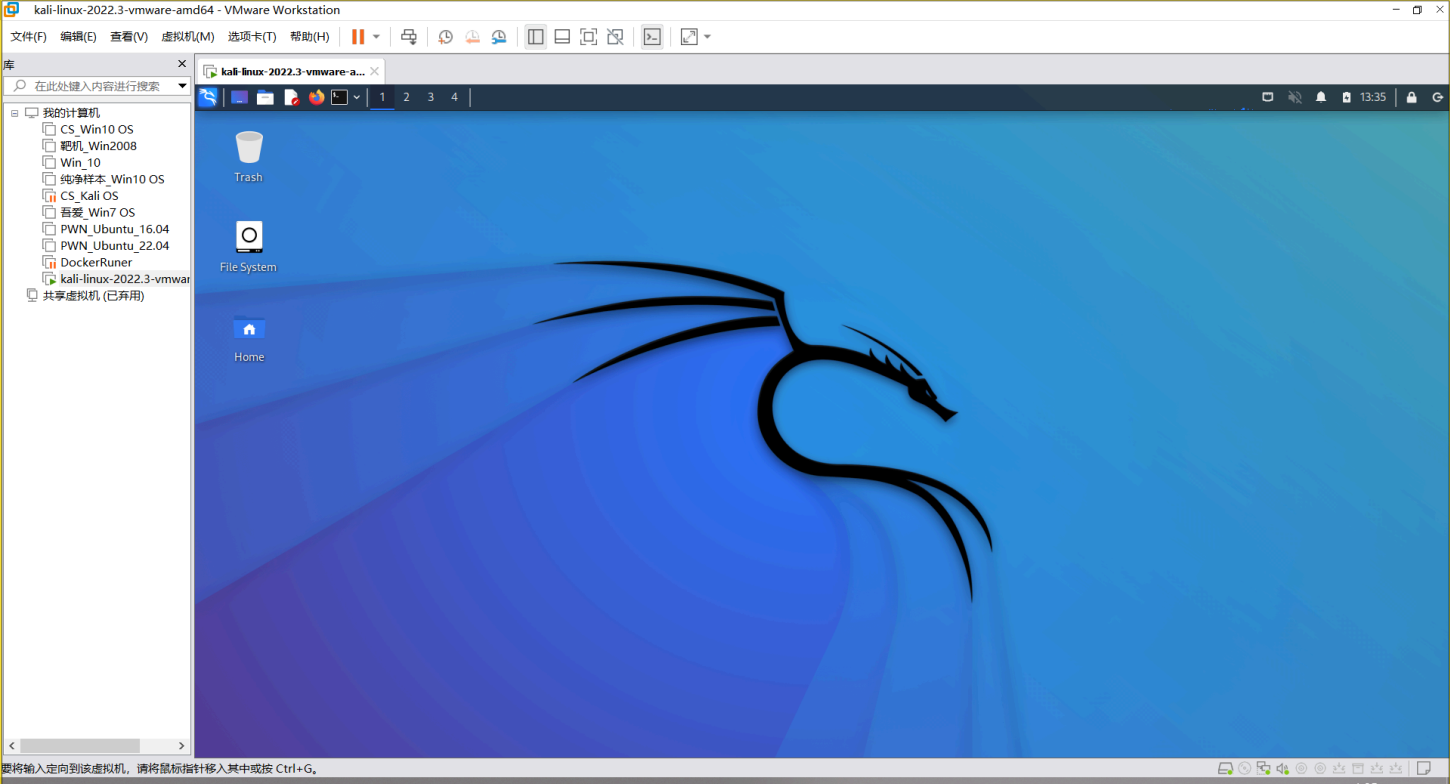


解压下载好的压缩包到您想要放置虚拟机的位置，然后双击配置文件直接打开自动添加，或者在 VMware 中手

动添加：



添加完成后，启动即可：



(当然，作为 Docker Runner，您可能需要设置一个稍微大点的内存值)

设备	摘要
内存	2 GB
处理器	4
硬盘 (SCSI)	80.1 GB
CD/DVD (IDE)	自动检测
网络适配器	NAT
USB 控制器	存在
声卡	自动检测
显示器	自动检测

内存

指定分配给此虚拟机的内存量。内存大小必须为 4 MB 的倍数。

此虚拟机的内存(M): 2048 MB

64 GB -

32 GB -

16 GB -

8 GB -

4 GB -

2 GB -

1 GB -

最大建议内存

(超出此大小可能发生内存交换。)

27.9 GB

Change Source?

笔者并不建议换源，Kali 官方源的速度还是不错的，换源可能会导致您软件管理出现问题。

Get Docker

Install

Docker 安装目前有两个版本，一个是官方的 **docker.ce** 版本，一个是有 Debian 团队维护的 **docker.io** 版本。

之前有争议说 **docker.io** 是旧版本的 Docker 而新版的 Docker 是 docker.ce

然而 docker.io 的版本有时会比 docker.ce 高，事实上，两者只是负责维护的组织不同，前者是 Debian 官方维护，后者则为 Docker 官方维护，并没后新旧之分：

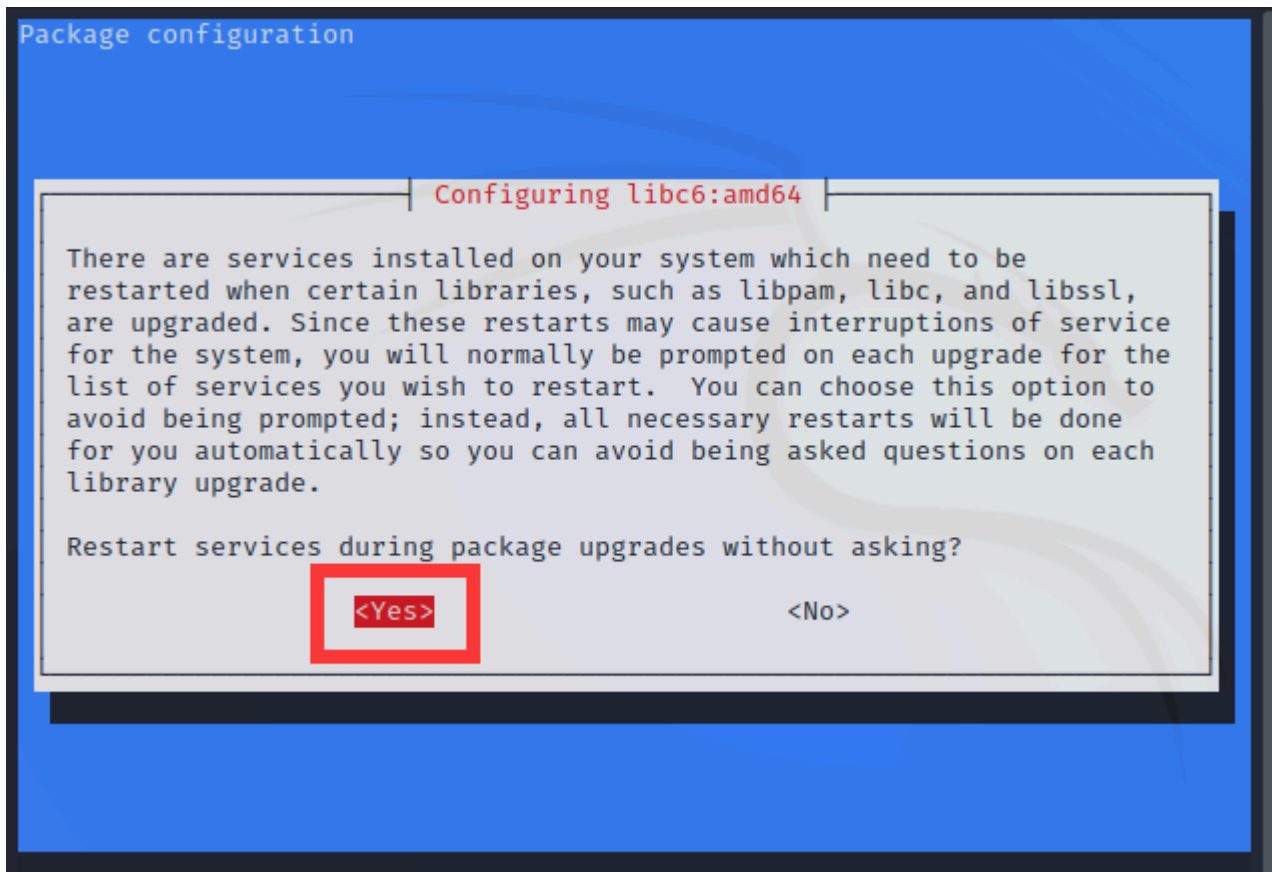
docker.io

该版本由 Debian 团队维护，采用 apt 的方式管理依赖
安装过程：

```
sudo apt-get update
sudo apt install docker.io
```

```
(kali㉿kali)-[~]  
$ sudo apt-get update  
[sudo] password for kali:  
Get:1 http://kali.download/kali kali-rolling InRelease [30.6 kB]  
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [18.7 MB]  
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [43.4 MB]  
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [110 kB]  
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [161 kB]  
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [232 kB]  
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [897 kB]  
Fetched 63.5 MB in 17s (3,726 kB/s)  
Reading package lists ... Done
```

```
(kali㉿kali)-[~]  
$ sudo apt install docker.io  
Reading package lists ... Done  
Building dependency tree ... Done  
Reading state information ... Done  
The following additional packages will be installed:  
  cgrouppfs-mount containerd criu libc-bin libc-dev-bin libc-devtools  
  libc-l10n libc6 libc6-dev libc6-i386 libintl-perl libintl-xs-perl  
  libmodule-find-perl libmodule-scandeps-perl libproc-processtable-perl  
  libsort-naturally-perl locales needrestart runc tini  
Suggested packages:  
  containernetworking-plugins docker-doc aufs-tools btrfs-progs debootstrap  
  rinse rootlesskit xfsprogs zfs-fuse | zfsutils-linux glibc-doc libnss-nis  
  libnss-nisplus  
The following NEW packages will be installed:  
  cgrouppfs-mount containerd criu docker.io libintl-perl libintl-xs-perl  
  libmodule-find-perl libmodule-scandeps-perl libproc-processtable-perl  
  libsort-naturally-perl needrestart runc tini  
The following packages will be upgraded:  
  libc-bin libc-dev-bin libc-devtools libc-l10n libc6 libc6-dev libc6-i386  
  locales  
8 upgraded, 13 newly installed, 0 to remove and 1033 not upgraded.  
Need to get 77.9 MB of archives.  
After this operation, 260 MB of additional disk space will be used.  
Do you want to continue? [Y/n] y
```



docker.ce

由 Docker 官方维护的版本，采用 go 的方式管理依赖，如果您要在开发 / 生产环境中使用 Docker，推荐使用该版本。

如果您的 Linux 版本为 Ubuntu/Debian，可以使用一下命令自动安装：

```
wget -qO- https://get.docker.com/ | sh
```

（注意，Docker 并不支持 Kali 一键安装，同样笔者也不推荐您用 Kali 作为开发生产环境，Kali 满足日常需要，使用 io 版本即可，当然您也可以使用 .ce 版本的手动安装）

手动安装：

可以参考 [Docker 官方的 Docker Engine 安装步骤](#)

但是该步骤在 Kali Linux 上面可能会存在一定问题，以 Kali 为例，完整的安装流程如下：

```
sudo apt-get update
sudo apt-get install \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

```
(kali㉿kali)-[~]
└─$ sudo apt-get install \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
ca-certificates is already the newest version (20211016).
The following additional packages will be installed:
  dirmngr gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client
  libcurl4 locales lsb-base sysvinit-utils
Suggested packages:
```

选项选择 Yes 就好。

然后按照官网提示，添加 Docker 官方的 GPG key：

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
```

然后设置仓库：

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

然后开始安装：

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

随后，如果您是 Kali Linux，您可能会遭遇以下报错：


```
(kali㉿kali)-[~]
└─$ sudo apt-get update
Hit:1 http://kali.download/kali kali-rolling InRelease
Ign:2 https://download.docker.com/linux/debian kali-rolling InRelease
Err:3 https://download.docker.com/linux/debian kali-rolling Release
  404 Not Found [IP: 108.138.7.88 443]
Reading package lists... Done
E: The repository 'https://download.docker.com/linux/debian kali-rolling Release' does not have a Release file.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
```

```
(kali㉿kali)-[~]
└─$ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package docker-ce is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

Package docker-ce-cli is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

E: Package 'docker-ce' has no installation candidate
E: Package 'docker-ce-cli' has no installation candidate
E: Unable to locate package containerd.io
E: Couldn't find any package by glob 'containerd.io'
E: Couldn't find any package by regex 'containerd.io'
E: Unable to locate package docker-compose-plugin
```

这是由于 Docker 官方并没有提供直接支持 Kali 的源，所以我们使用 Debian 的源就好：

```
printf "%s\n" "deb [arch=amd64] https://download.docker.com/linux/debian buster stable" | \
sudo tee /etc/apt/sources.list.d/docker-ce.list
```

并且添加对应的密钥：

```
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo apt-key add -
```

注意不要忘了给密钥相应权限：

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

此时再跑一次 `sudo apt-get update`：

您可能会遇到一个找不到的源，这个是正常的。

```
(kali㉿kali)-[~]
└─$ sudo apt-get update
Hit:1 http://kali.download/kali kali-rolling InRelease
Ign:2 https://download.docker.com/linux/debian kali-rolling InRelease
Get:3 https://download.docker.com/linux/debian buster InRelease [54.0 kB]
Err:4 https://download.docker.com/linux/debian kali-rolling Release
  404 Not Found [IP: 52.222.214.63 443]
Get:5 https://download.docker.com/linux/debian buster/stable amd64 Packages [28.5 kB]
Get:6 https://download.docker.com/linux/debian buster/stable amd64 Contents (deb) [2,040 B]
Reading package lists... Done
W: https://download.docker.com/linux/debian/dists/buster/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details.
E: The repository 'https://download.docker.com/linux/debian kali-rolling Release' does not have a Release file.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
```

接下来安装 Docker：

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

就能正常获取了：

```
(kali㉿kali)-[~]
$ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following additional packages will be installed:
  docker-ce-rootless-extras docker-scan-plugin libslirp0 pigz slirp4netns
Suggested packages:
  aufs-tools cgroupfs-mount | cgroup-lite
The following NEW packages will be installed:
  containerd.io docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin docker-scan-plugin libslirp0 pigz slirp4netns
0 upgraded, 9 newly installed, 0 to remove and 1032 not upgraded.
Need to get 112 MB of archives.
After this operation, 442 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

安装完成后，运行 hello-world 容器，得到图示则安装成功。

```
sudo docker run hello-world
```

```
(kali㉿kali)-[~]
$ sudo docker run hello-world
[sudo] password for kali:

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Docker Hub

尝试登录您的 **Docker Hub** 账号：

```
(kali㉿kali)-[~]
$ sudo docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don
't have a Docker ID, head over to https://hub.docker.com to create one.
Username: probius
Password:
Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http:
request canceled while waiting for connection (Client.Timeout exceeded while
awaiting headers)
```

如果出现类似错误，请手动配置 DNS：

```
sudo vim /etc/resolv.conf
```



```
File Actions Edit View Help
# Generated by NetworkManager
search localdomain
nameserver 114.114.114.114
~
~
```

保存，再次尝试即可成功登录：

```
(kali㉿kali)-[~]
$ sudo docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't
have a Docker ID, head over to https://hub.docker.com to create one.
Username: probius
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.js
on.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Build SSH + VSCode + Docker Workflow

该步骤适用于支持 SSH 协议的 Linux ~~（废话，哪个Linux没有个SSH）~~，除了在本地的 Kali 或者其他 Linux 上面构建，也支持远程构建该工作流。

Start SSH Service

Kali Linux

Kali 虚拟机默认是没有开启 SSH 服务的（SSH 状态：`/etc/init.d/ssh status`）

```
(kali㉿kali)-[~]
$ /etc/init.d/ssh status
o ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; disabled; vendor preset
: disabled)
   Active: inactive (dead)
     Docs: man:sshd(8)
           man:sshd_config(5)
```

所以需要手动启动一下：

```
sudo /etc/init.d/ssh start
```

当然，为了方便，我们将 ssh 添加到开机自启中：

```
sudo update-rc.d ssh enable
# Or use:
systemctl enable ssh.service
```

```
(kali㉿kali)-[~]
$ sudo update-rc.d ssh enable
```

启动之后就可以开始后面的步骤了。

Ubuntu

由于 Ubuntu 默认不自带 openssh-server，所以这里需要手动安装。安装 openssh-server：

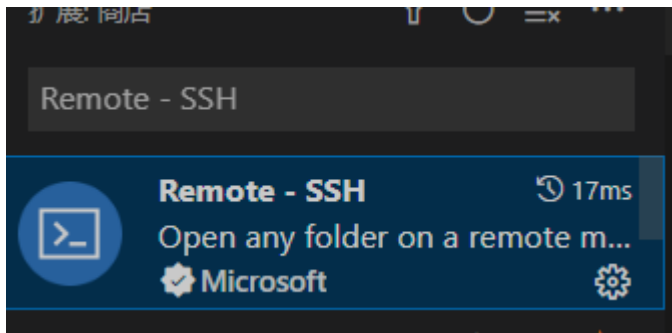
```
sudo apt-get install openssh-server
```

安装完成后启动服务：

```
sudo service ssh start
```

Get Remote - SSH Extension

为了使用 VSCode 的远程 SSH 功能 您可能需要在扩展中安装 Remote - SSH 插件：

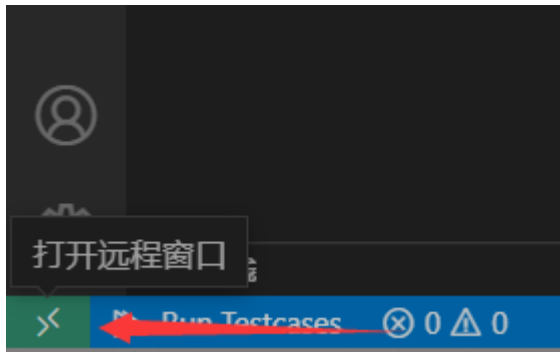


Let VSCode Connect to Host

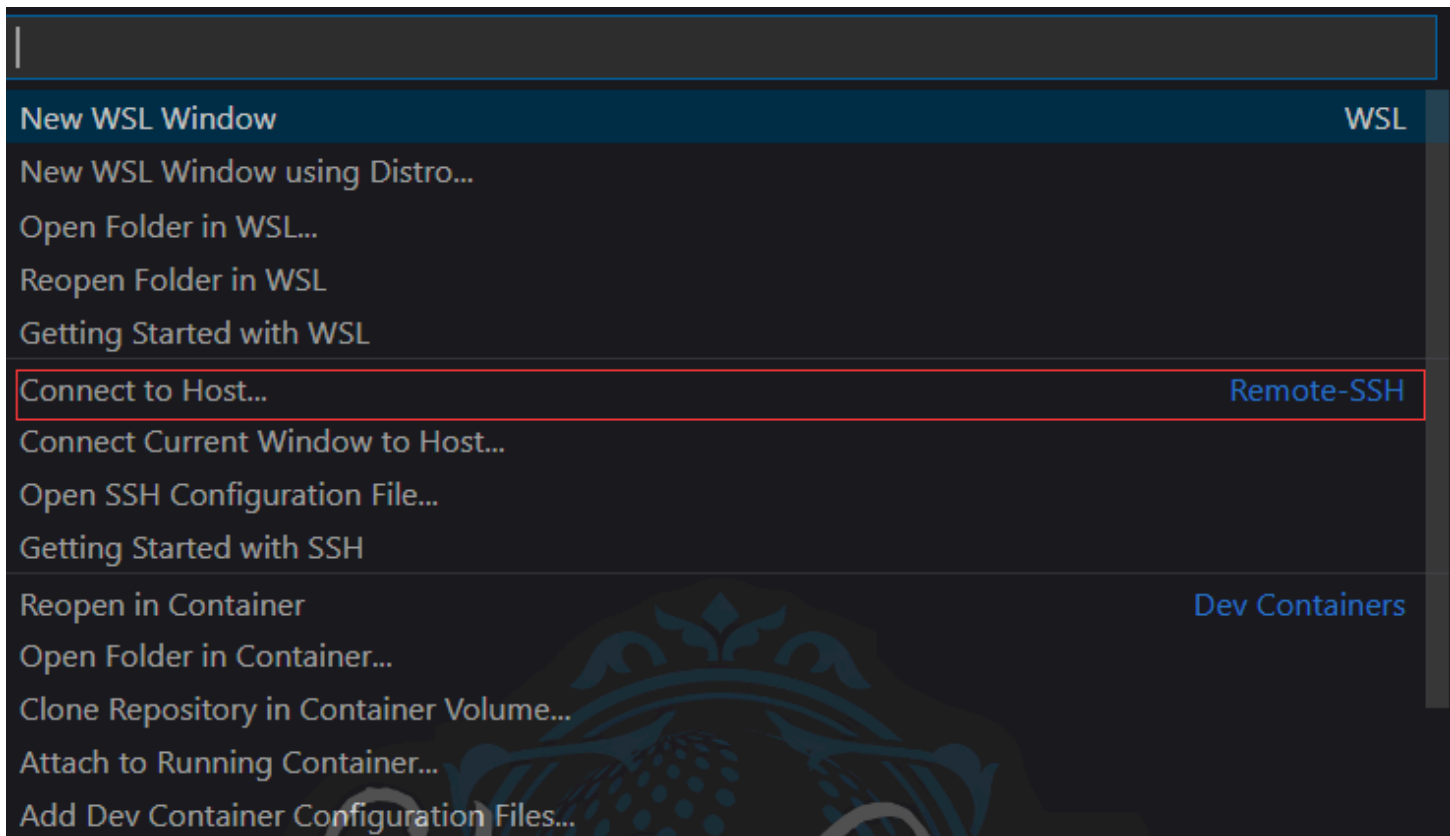
连接方式有两种：密码连接 和 SSH的公私钥。一般情况下我们使用密码连接，这样的方法安全性较低，而且每次连接和每次切换目录的时候都会要求输入密码，所以在确保能够密码连接情况下，我们可以进一步使用公私钥验证的方式来提升安全性和优化操作友好性。

Using Password Connection

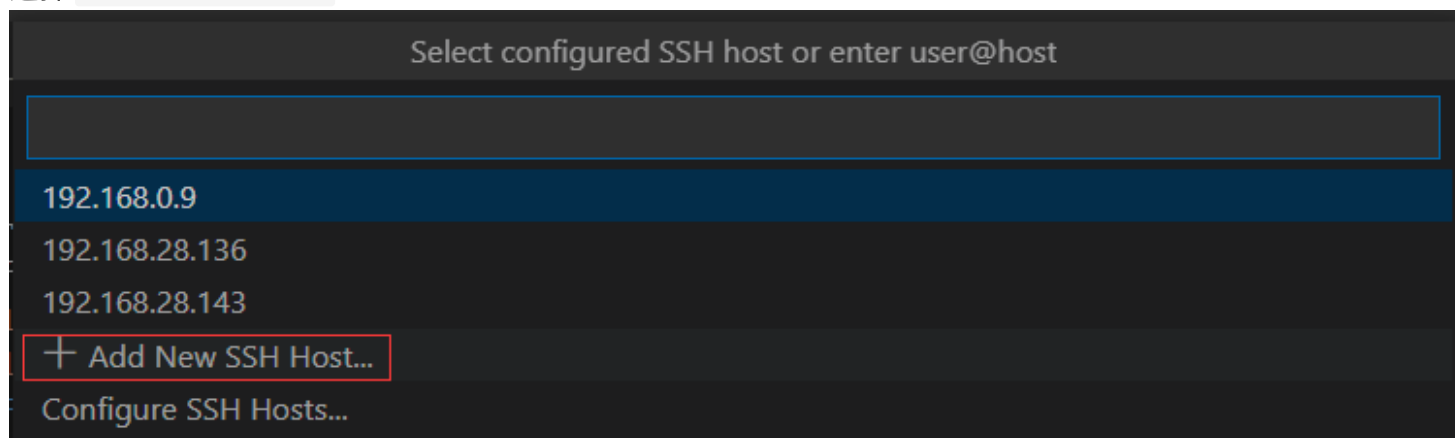
打开 VSCode，点击左下角的远程连接（绿色的部分）



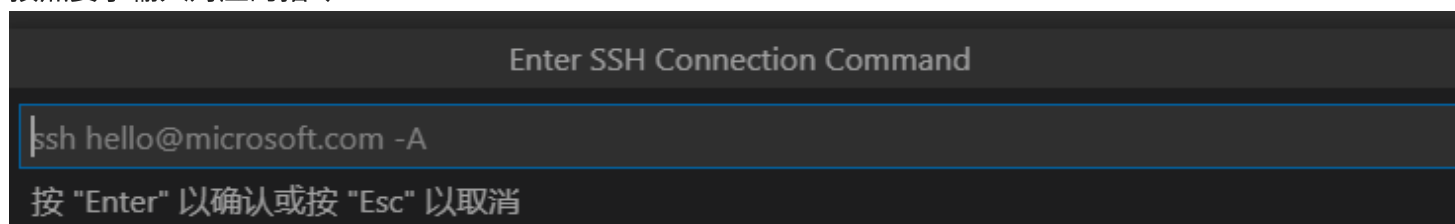
接着在弹出的窗口中选择 Connect to Host



选择 Add New SSH Host :



按照要求输入对应的指令：

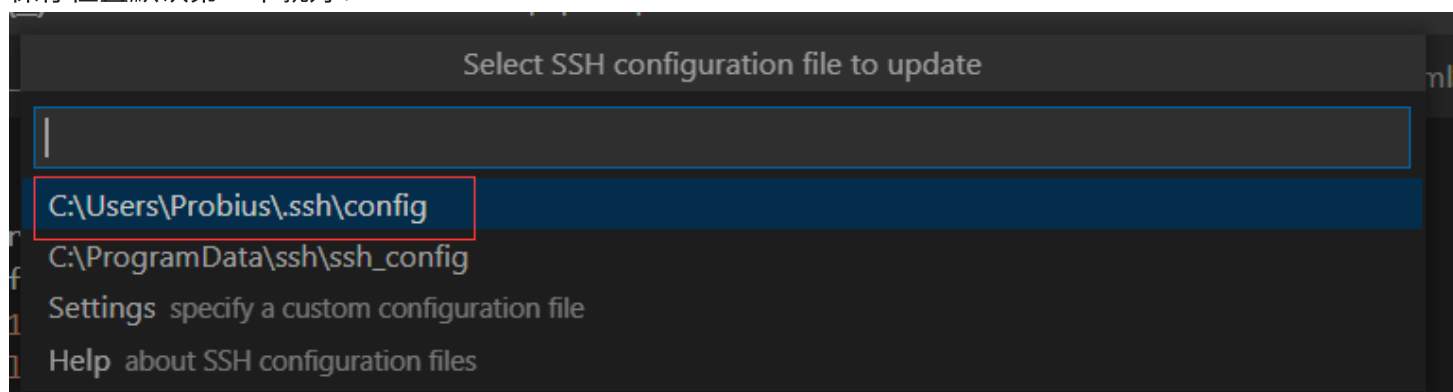


IP 可通过 `ip a` 获取

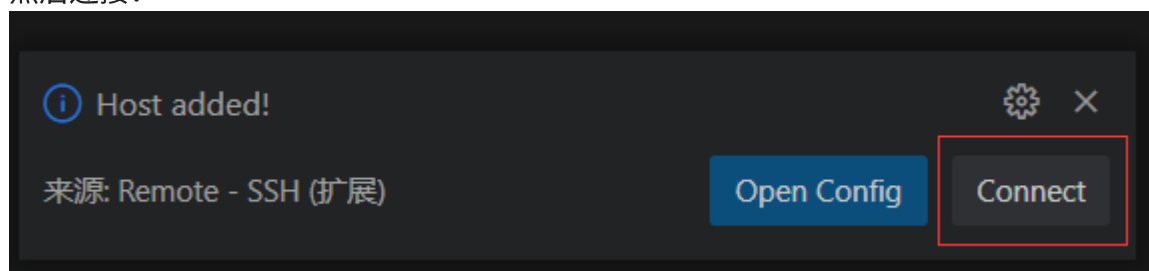
```
(kali㉿kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:02:80:bf brd ff:ff:ff:ff:ff:ff  
    inet 192.168.28.145/24 brd 192.168.28.255 scope global dynamic noprefixroute eth0  
        valid_lft 1245sec preferred_lft 1245sec  
    inet6 fe80::5f81:be6b:20be:9ba5/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default  
    link/ether 02:42:58:d5:c6:4a brd ff:ff:ff:ff:ff:ff  
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0  
        valid_lft forever preferred_lft forever
```

```
ssh kali@192.168.28.145 -A
```

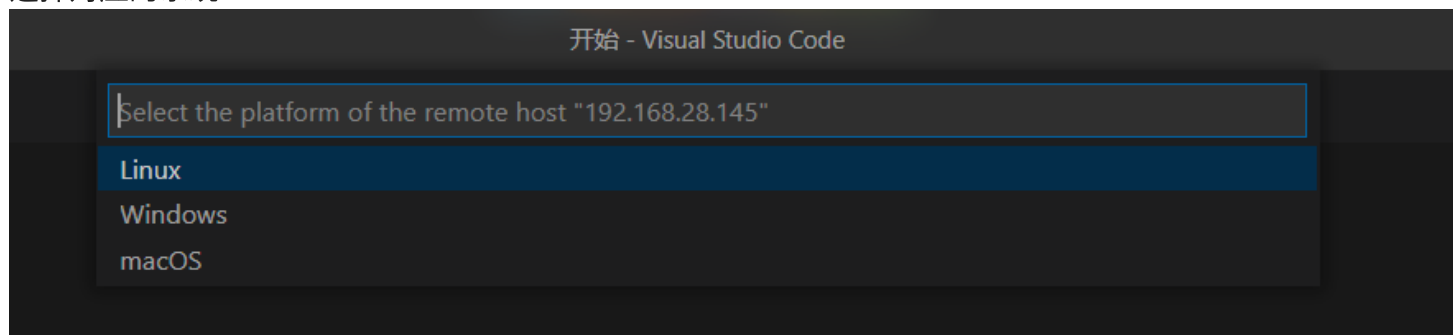
保存位置默认第一个就好：



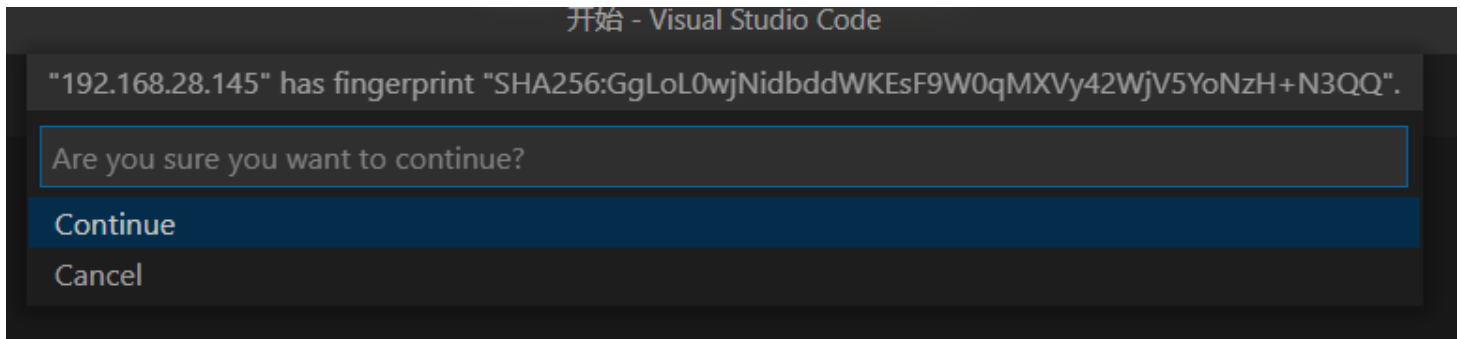
然后连接：



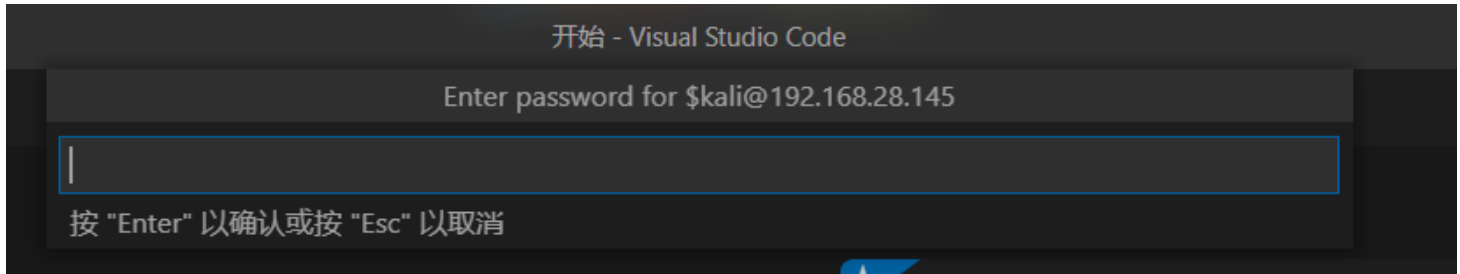
选择对应的系统：



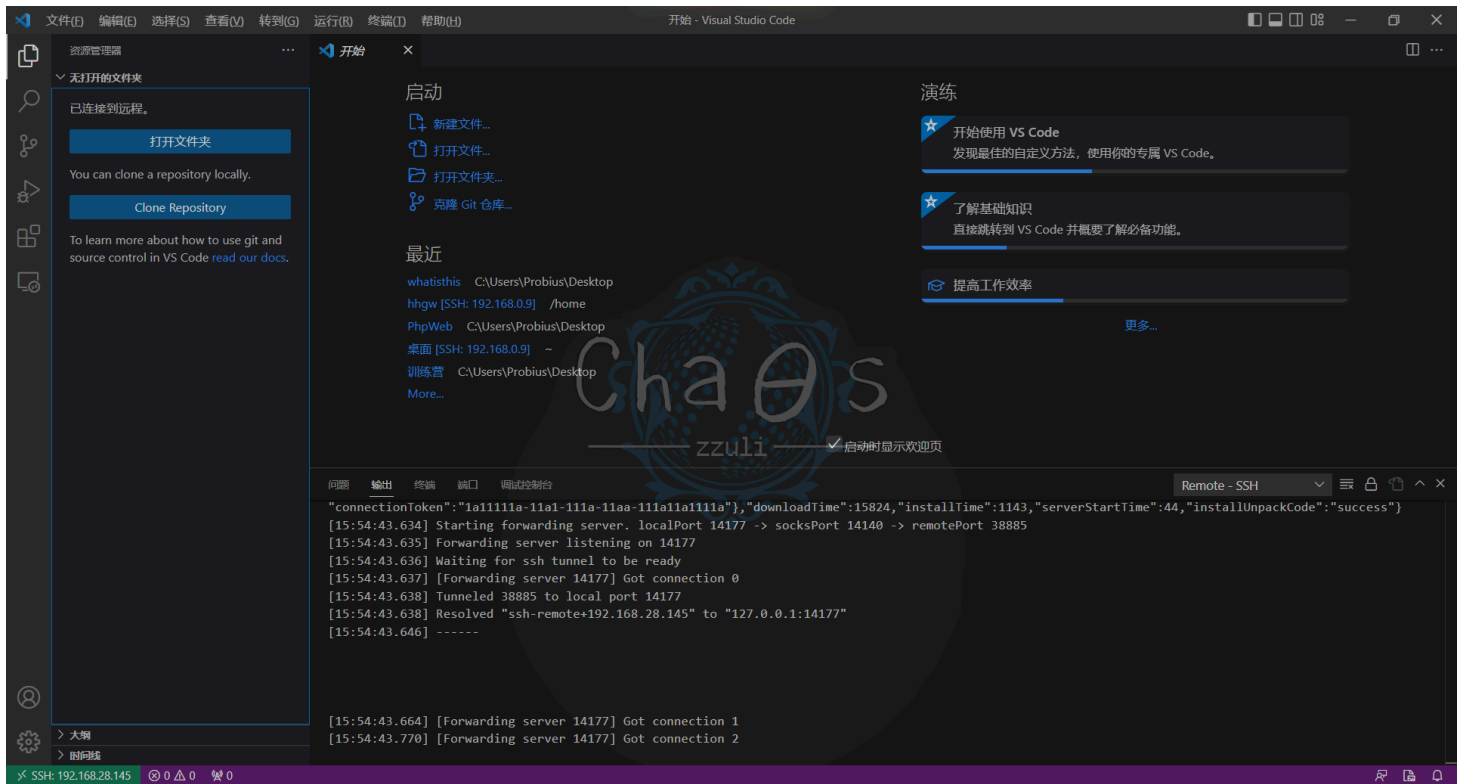
选择继续：



输入密码：



初始化完成后就连接上了：



Using Public and Private Keys

CREATE PUBLIC AND PRIVATE KEYS

首先在本机上生成对应的公钥私钥：

```
ssh-keygen
```

```
● PS C:\Users\Probius\Desktop\key> ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (C:\Users\Probius\.ssh\id_rsa): Kali_test
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in Kali_test.
Your public key has been saved in Kali_test.pub.
The key fingerprint is:
SHA256:AzwXs9DVqHXLWUoj2+S2qKGpqdA3fT565FBxoT6D/Tw probius@ProbiusOfficial
The key's randomart image is:
+---[RSA 3072]-----+
|      ..0.o.=      |
|      . .+=B =     |
|      + +% = o     |
|      B+ = o       |
|      ..So .       |
|      .o o*.       |
|      . . oo.*. E   |
|      oo..o+ .     |
|      ..o. .o..    |
+---[SHA256]-----+
● PS C:\Users\Probius\Desktop\key>
```

Enter file in which to save the key (C:\Users\Probius\.ssh\id_rsa):

输入保存路径，如果为空默认为 `C:\Users\USERNAME\.ssh\id_rsa`，这里笔者直接输入了名字，让其以该名字直接保存在当前目录

Enter passphrase (empty for no passphrase):

输入私钥密码，为空则没有

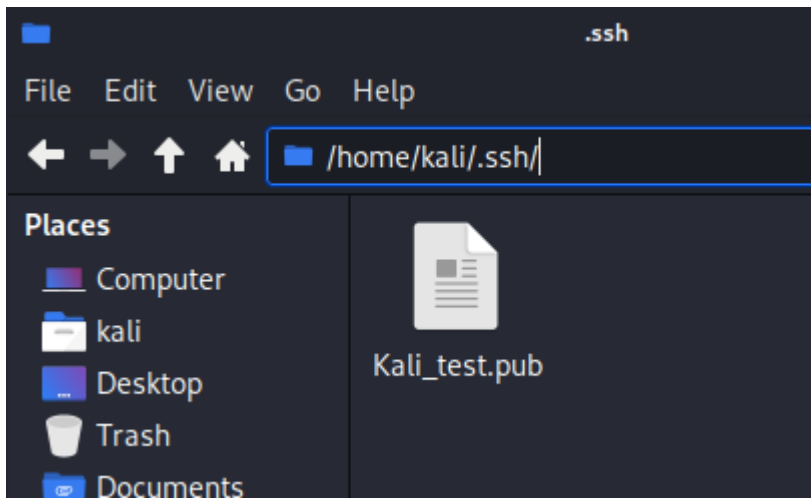
Enter same passphrase again:

注意：如果您使用了私钥密码，那么在进行 ssh 登录的时候，会提示您输入私钥密码，相当于再使用公私钥登录的情况下又加了一层密码，安全性有很大的提升，但是这样会变得和密码登录一样每次都要输入，这里笔者建议，如果是本地一路回车就好了，如果是远程资源的管理，可以综合安全性考虑

生成过后，得到的 `id_rsa.pub` (`Kali_test.pub`) 是 **公钥**，`id_rsa` (`Kali_test`) 是 **私钥**
将得到的私钥放在我们的 `.ssh`(`C:\Users\USERNAME\.ssh\`) 中。
(您也可以在生成的时候就让他放置在此，当然这并不是重点)

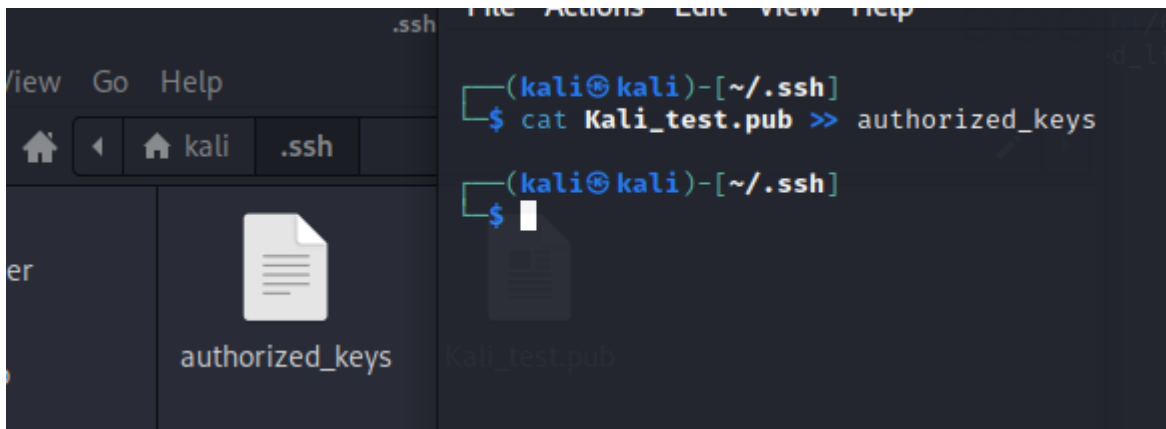
CONFIGURE REMOTE HOST

将得到的公钥上传到 Kali/ 服务器中对应用户的 `.ssh` 文件夹中 (请确保该文件夹至少有 700 的权限)： (注意 Kali 默认没有该文件夹，需要用户手动创建，并且赋 700 权限 `mkdir ~/.ssh`)



在此处打开终端，用公钥文件来生成 authorized_keys：

```
cat id_rsa.pub >> authorized_keys
# 此处即 cat Kali_test.pub >> authorized_keys
```



由于 Linux 系统对 .ssh 拥有比较严格的权限限制，请确保权限配置正确：

```
sudo chmod 600 authorized_keys
sudo chmod 700 ~/.ssh
```

编辑 ssh 配置文件

```
sudo vim /etc/ssh/sshd_config
```

确保拥有以下条目：

```
RSAAuthentication yes
PubkeyAuthentication yes
```

注意 Kali 的 SSH 默认配置是没有上述两项的，需要自行添加：

```
# ForceCommand cvs server
RSAAuthentication yes
PubkeyAuthentication yes
~
```

然后搞定一切后，重启 SSH 服务：

```
sudo service ssh restart
```

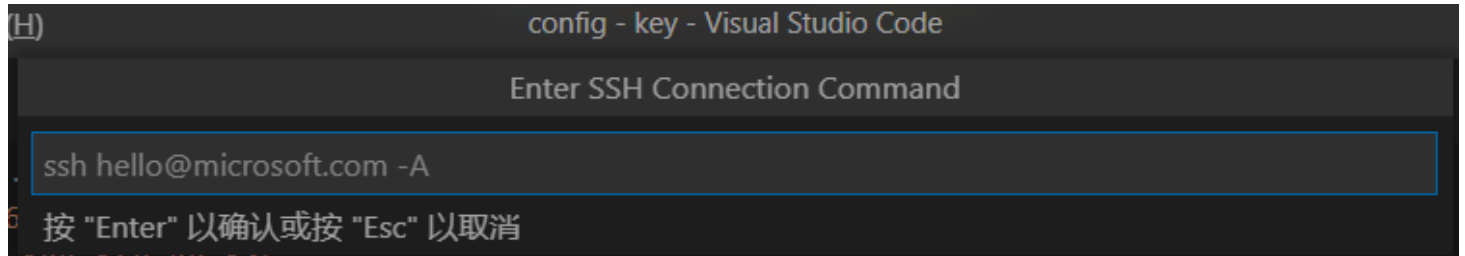
Linux 部分的配置这样就搞定啦~

(在您确保您的连接无误后，如果是服务器等远程设备，可以在 config 中关闭密码登录以提升安全性：
PasswordAuthentication no)

CONFIGURE LOCAL HOST

然后是 Windows，也就是 VSCode 这边的配置：

添加 New Host：



使用：

```
ssh USERNAME@IP -i C:\\Users\\USERNAME\\.ssh\\RSA_ID
```

(注意使用指令添加时，务必使用双斜杠)

当然，也可以在已经添加的配置文件上改：

```
Host IP
  HostName HOSTNAME
  User USERNAME
  IdentityFile C:\\Users\\USERNAME\\.ssh\\Kali_test
```

(根据自己的实际配置修改 IP、HOSTNAME、USERNAME 值)

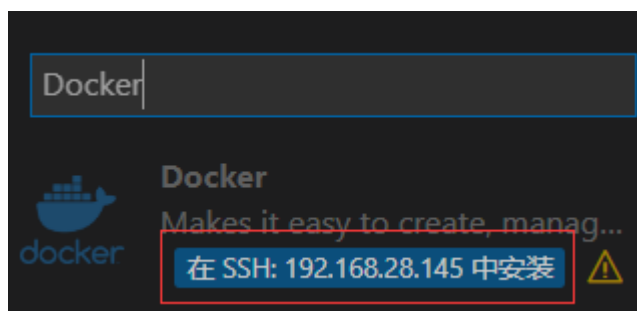
如果操作无误，且没有设置私钥密码，那么点击连接之后无需其他额外操作即可连接；如果设置了私钥密码，那么按照提示输入即可。

OTHER - ABOUT SSH KEY LOGIN

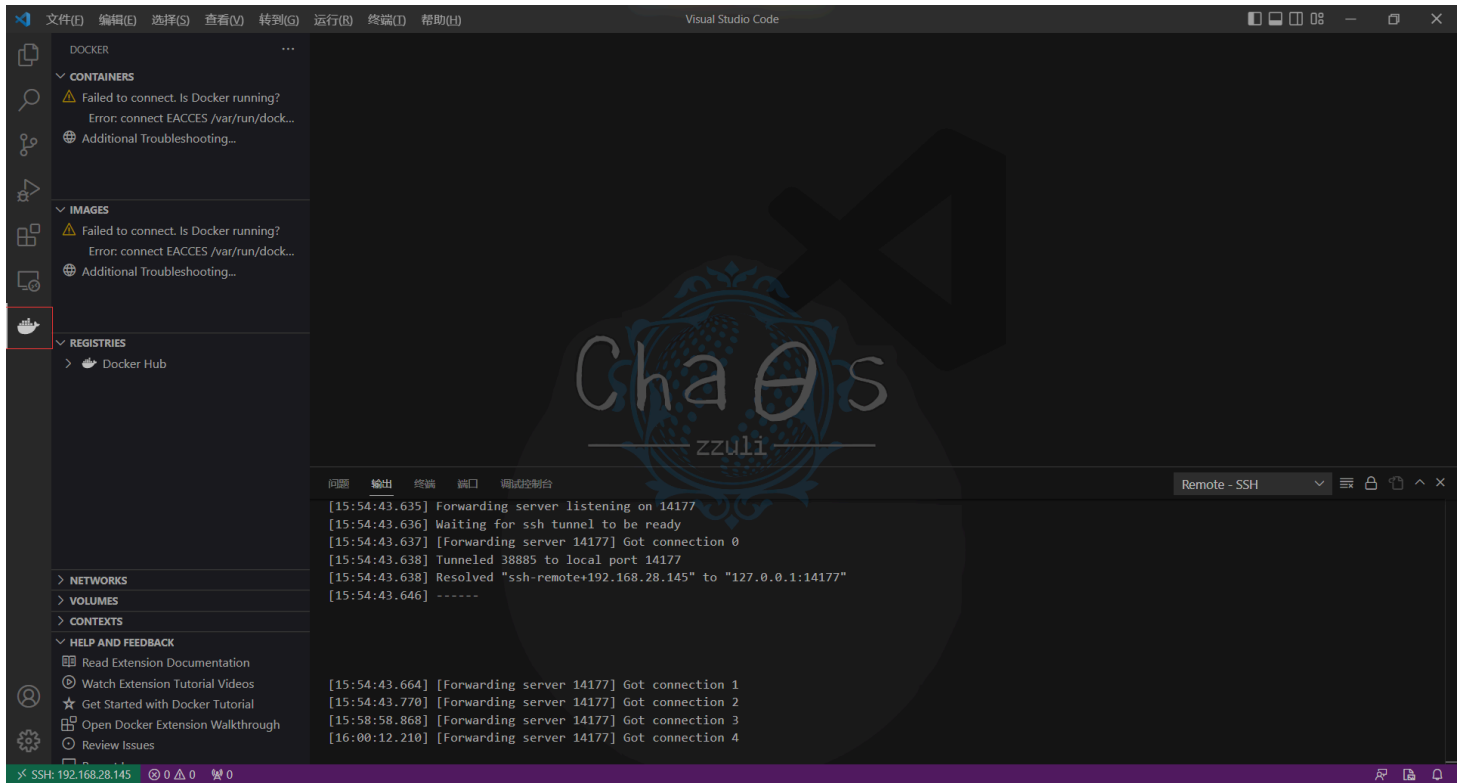
关于 SSH 密钥登录的更多知识可以看这一篇文章：[SSH 密钥登录](#)

Get Docker Extension

Install

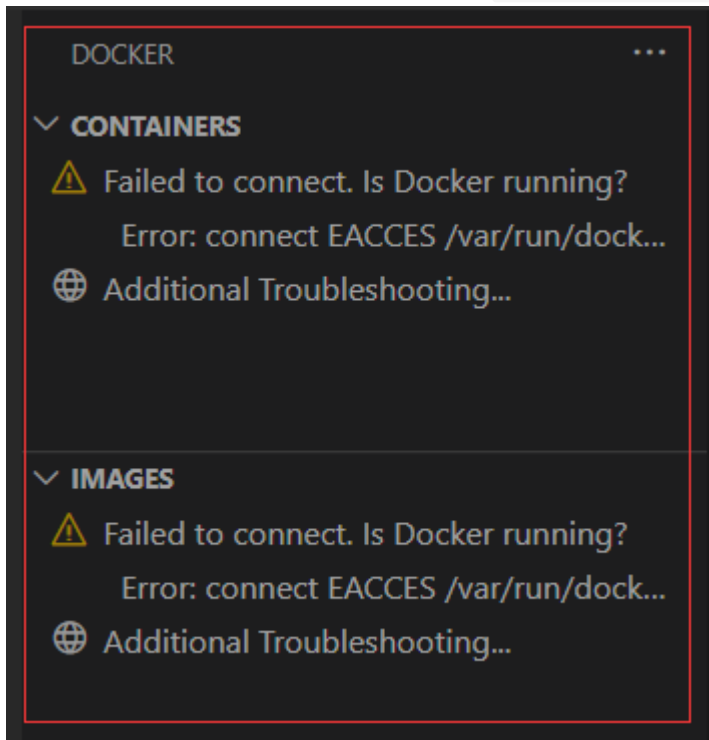


(当然 如果显示的是 安装 也是一样的)



Common Problem Solution

如果您安装 Docker 时使用自动安装，应该不会遇到该问题，自动安装的脚本会自动完成用户组添加的操作。由于权限原因，我们可能无法访问到 `/var/run/docker.sock`



BY ADDING USER GROUP

一劳永逸的解决权限问题，将当前用户组添加到 Docker 组即可。创建 Docker 用户组，默认安装时就会自动创建，所以执行可能会显示 `exist`

```
sudo groupadd docker
```

添加当前用户到该组：



```
sudo usermod -aG docker ${USER}
```

然后重新启动容器服务：

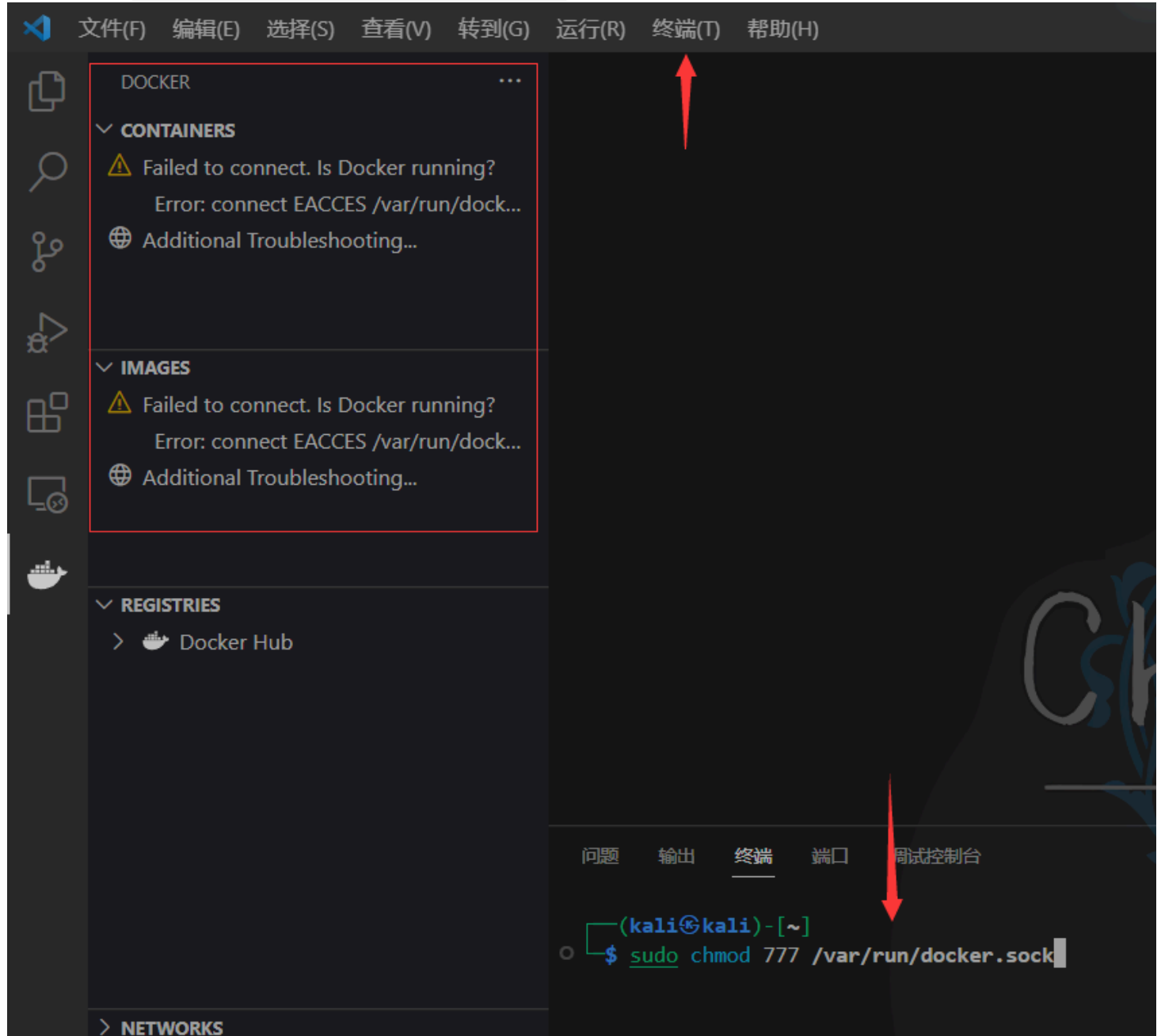
```
sudo systemctl restart docker
```

再次重新登入即可，如果重新登录之后依旧报错无法读取，请尝试重启：

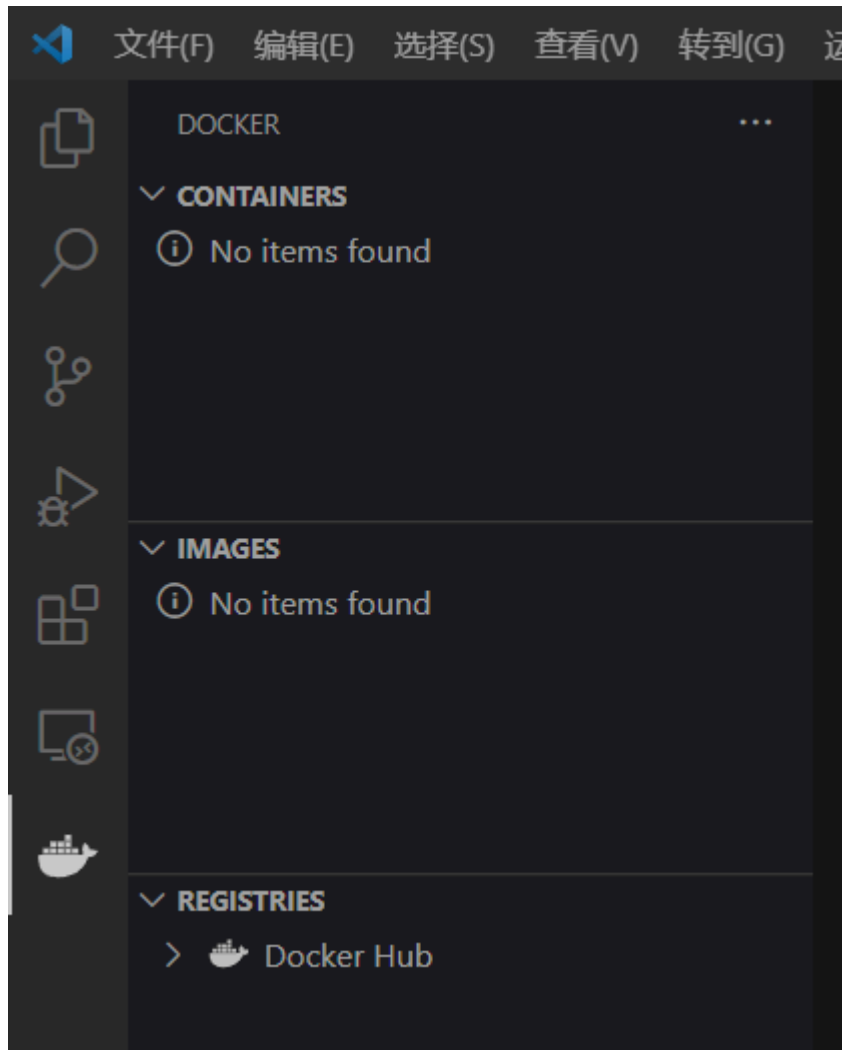
```
reboot
```

BY CHMOD COMMAND

也可以使用 `sudo chmod 777 /var/run/docker.sock`，但是每次重启之后权限都会重置，不推荐

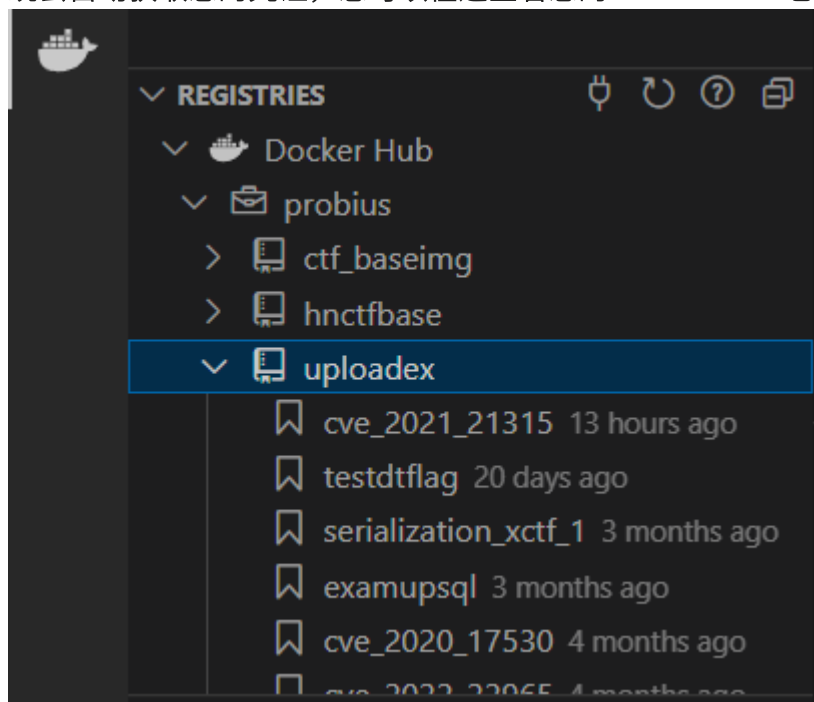


然后就能正常访问了：



Start Using Docker

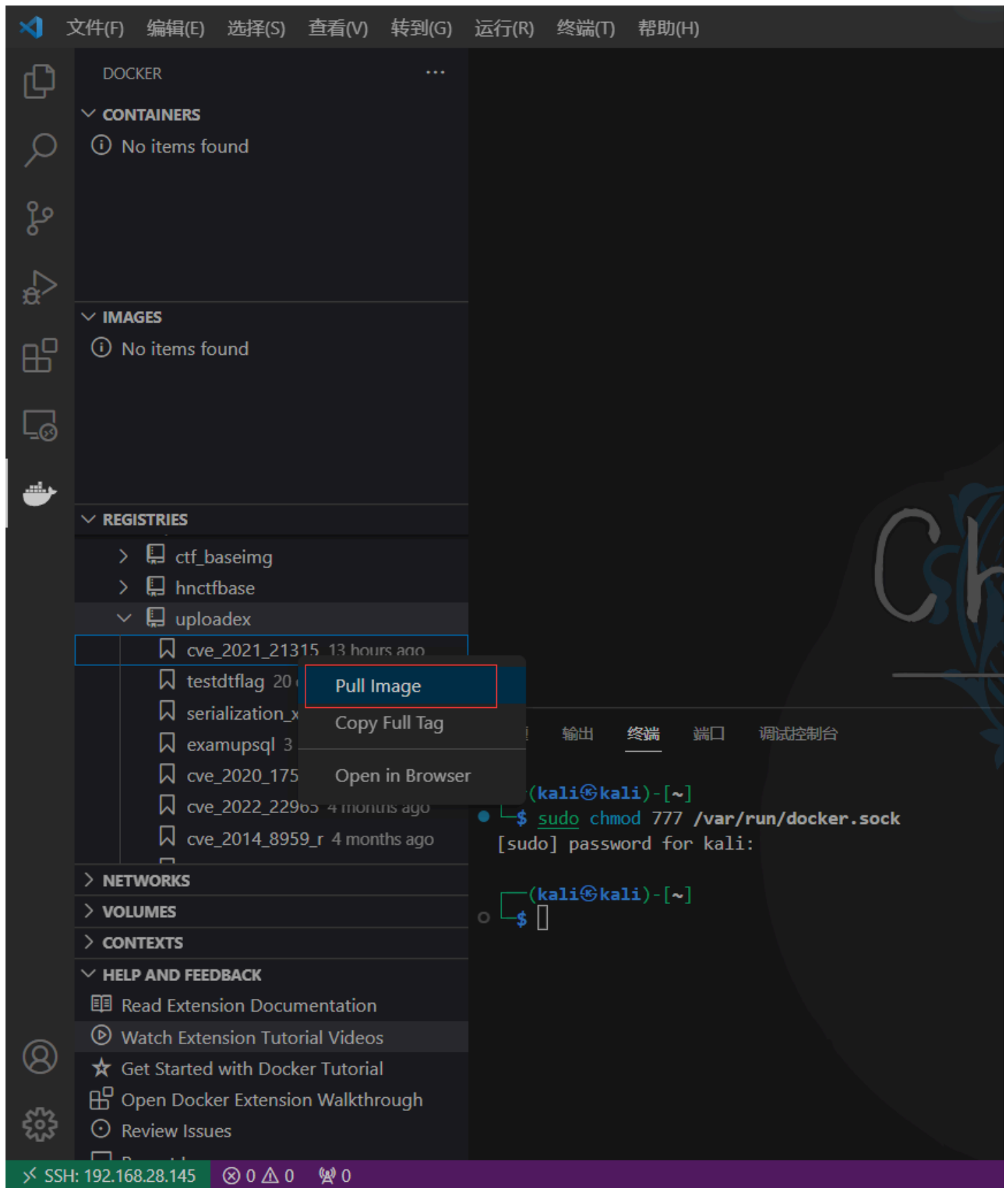
（以下操作使用 **Docker Hub** 演示）因为在安装 Docker 的时候登录过 Docker Hub，所以这里连接之后，系统会自动获取您的凭证，您可以在这里查看您的 Docker Hub 仓库：



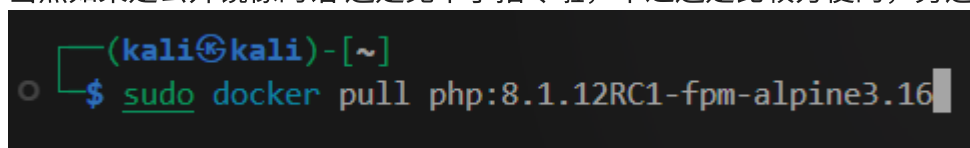
当然，您也可以添加其他仓库，具体的用法可以参考官方文档：

Pull Images

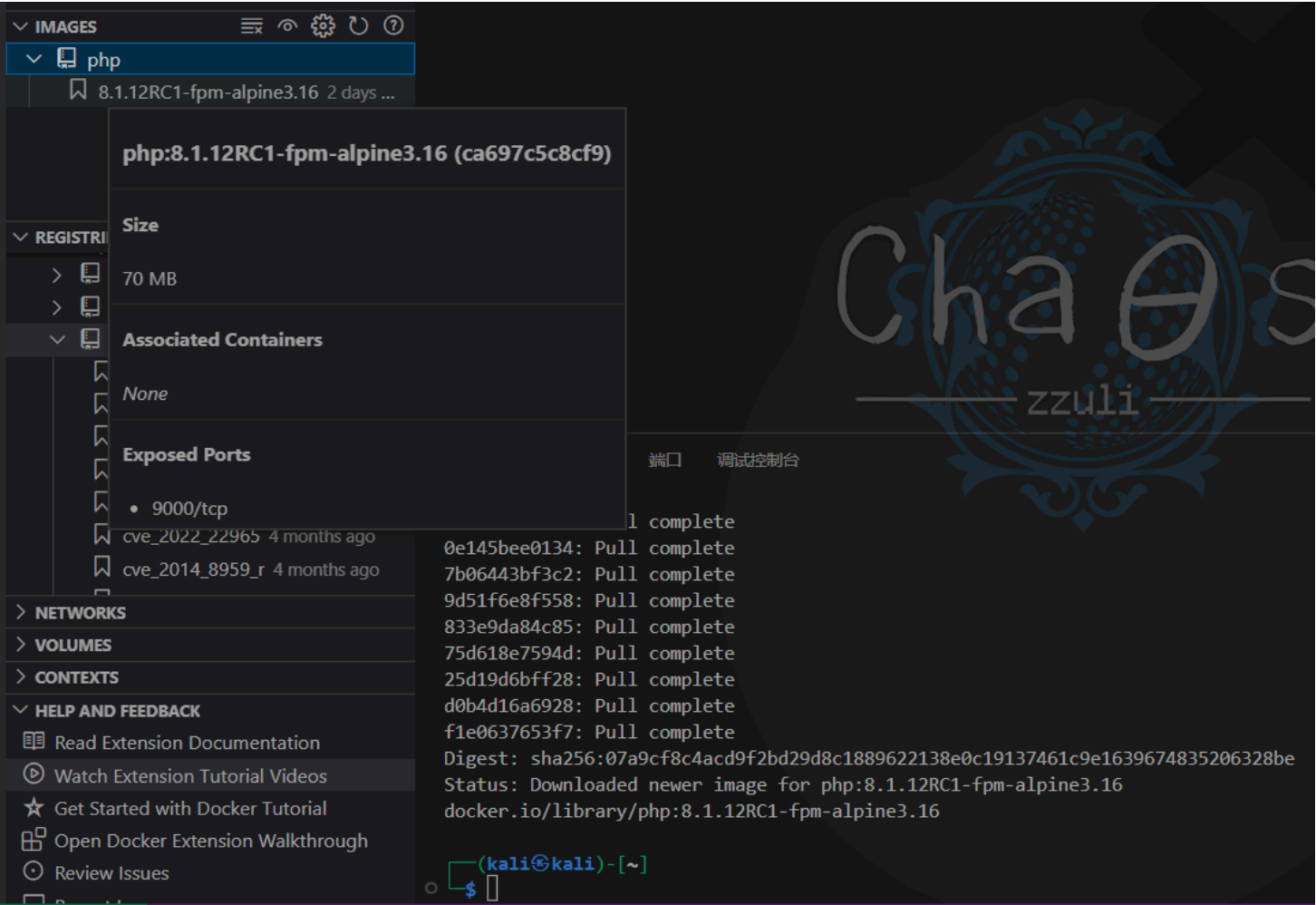
当然 对于自己仓库的镜像还是比较方便的 233:



当然如果是公开镜像的话 还是免不了指令啦，不过还是比较方便的，旁边就是终端：



完成之后就能在 IMAGES 栏看到了：

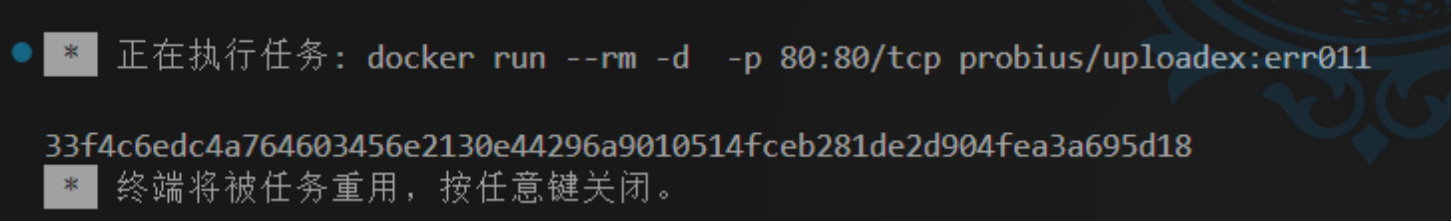


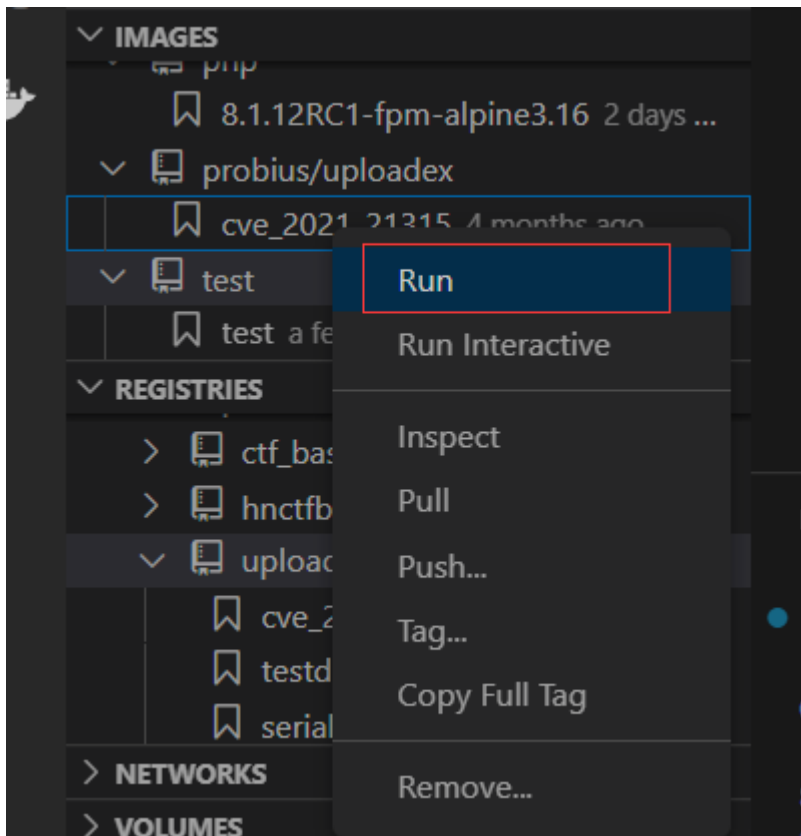
Run Docker

运行容器有两个选项 Run / Run Interactive

RUN

默认后台运行，Run 执行后效果和 Linux 执行 -d 的效果相同：





RUN INTERACTIVE

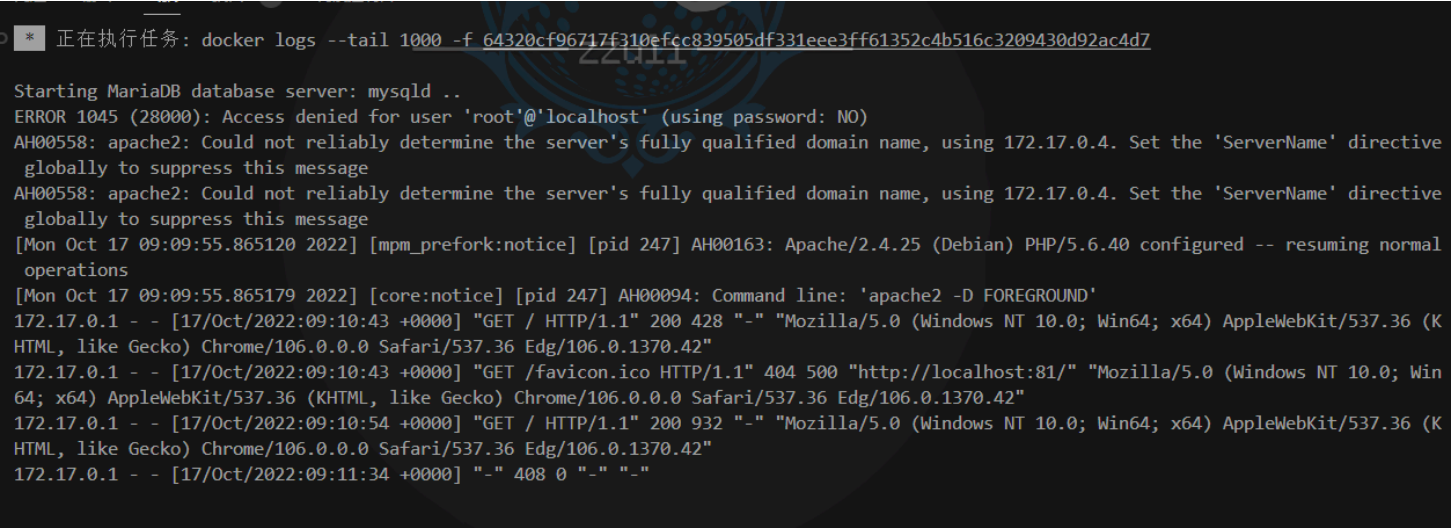
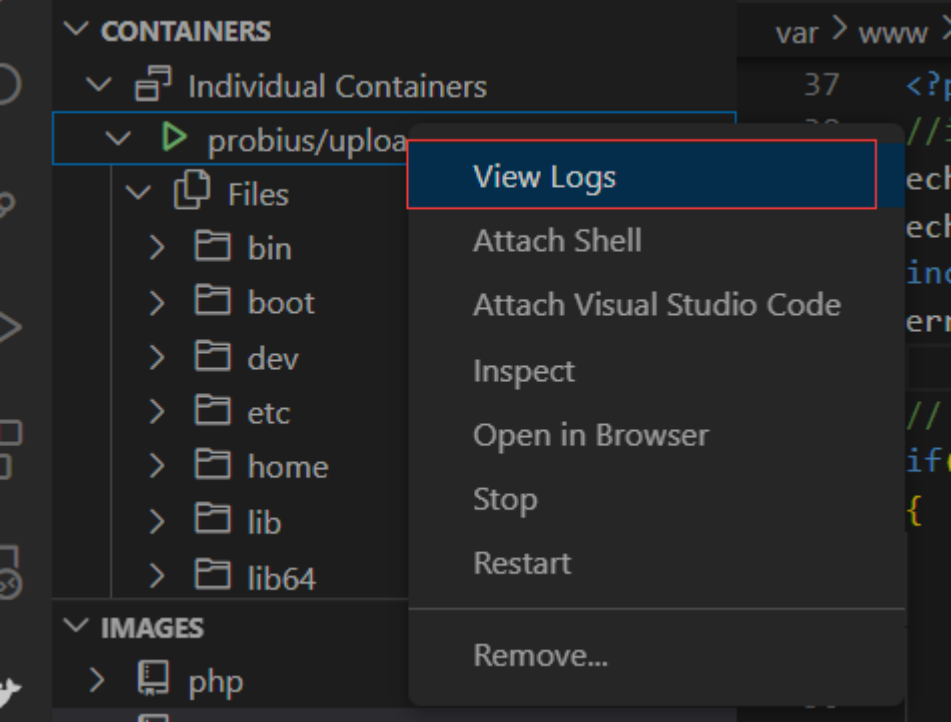
该选项运行执行后，除了运行容器，还会跳到容器的汇总 shell 处，这里会实时打印容器日志，效果和 View Log 效果相同：

```
正在执行任务: docker logs --tail 1000 -f 33f4c6edc4a764603456e2130e44296a9010514fceb281de2d904fea3a695d18

Starting MariaDB database server: mysqld ..
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: NO)
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
[Tue Oct 18 17:58:19.673005 2022] [mpm_prefork:notice] [pid 246] AH00163: Apache/2.4.25 (Debian) PHP/5.6.40 configured -- resuming normal operations
[Tue Oct 18 17:58:19.673049 2022] [core:notice] [pid 246] AH00094: Command line: 'apache2 -D FOREGROUND'
```

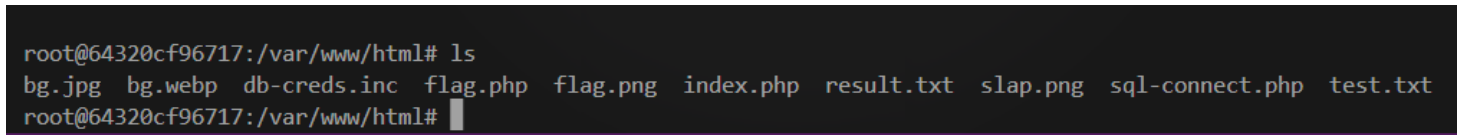
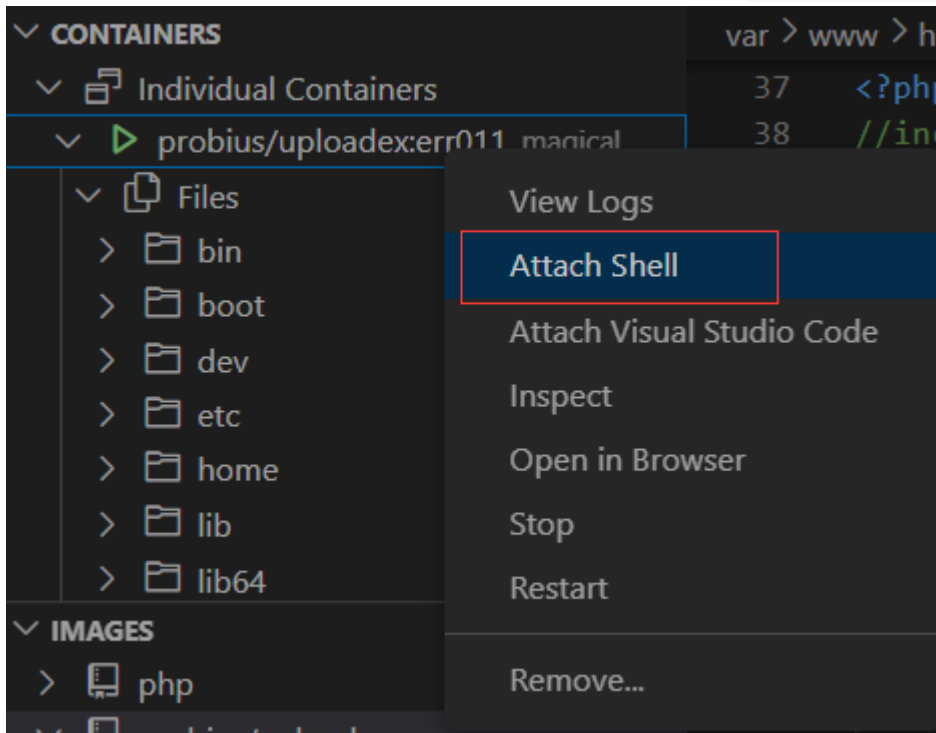
View Log

直达容器后台日志，如果在运行的时候选择 Run Interactive 那么也会跳转到该 log 页面。



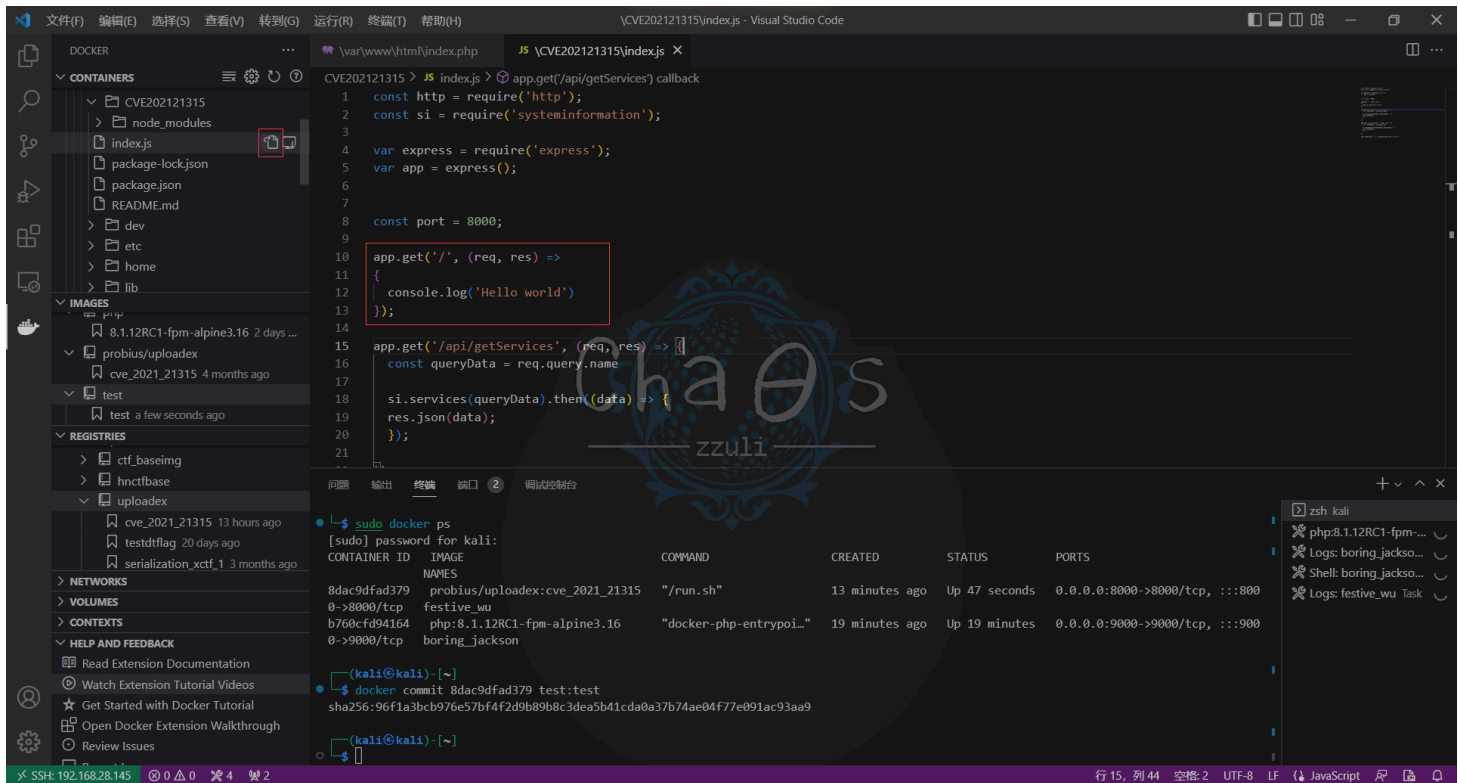
Attach Shell

右键选中之后可以直接建立一个容器内部的 shell，省去 `docker exec -i -t ID /bin/bash`

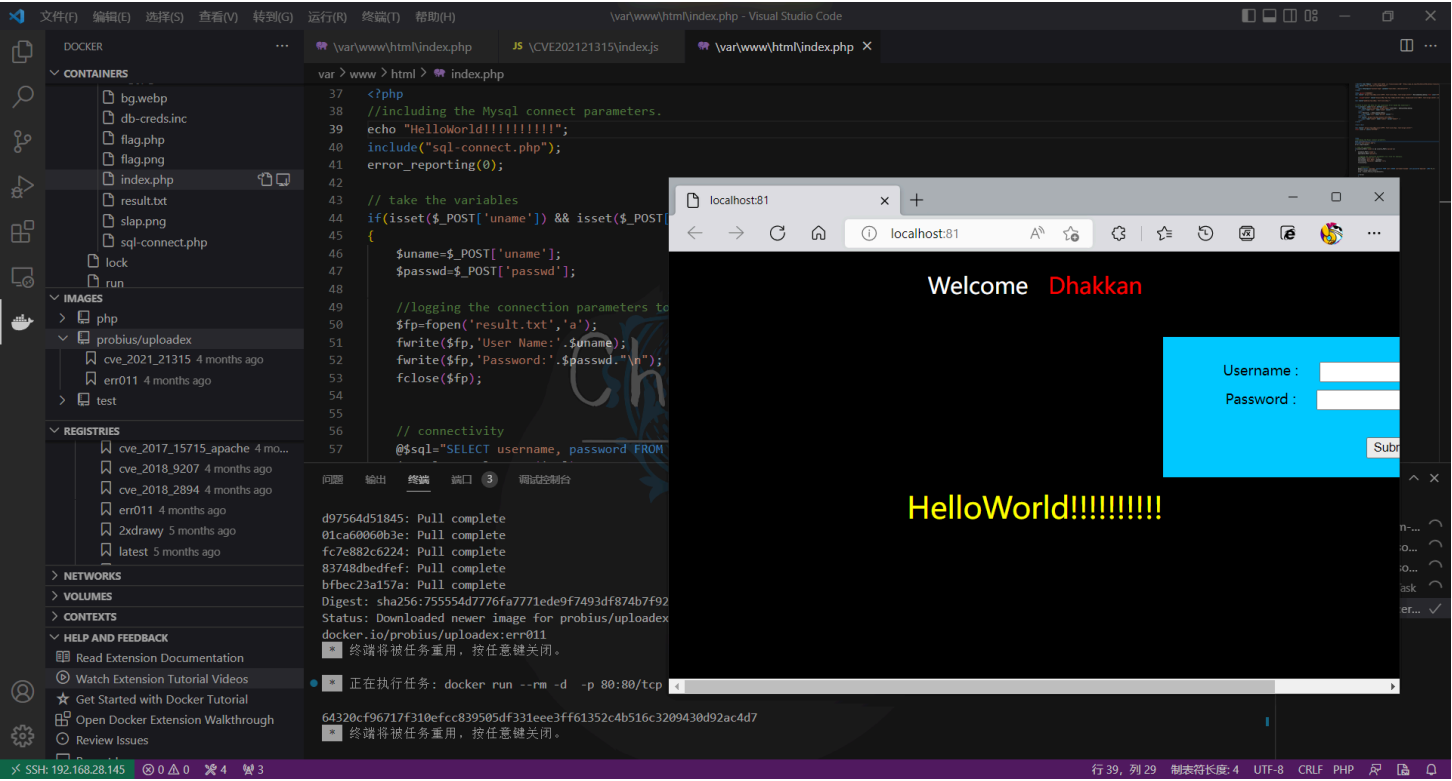


Edit Container

在 CONTAINERS 中下拉一个容器的 file 可以对其进行查看和编辑：

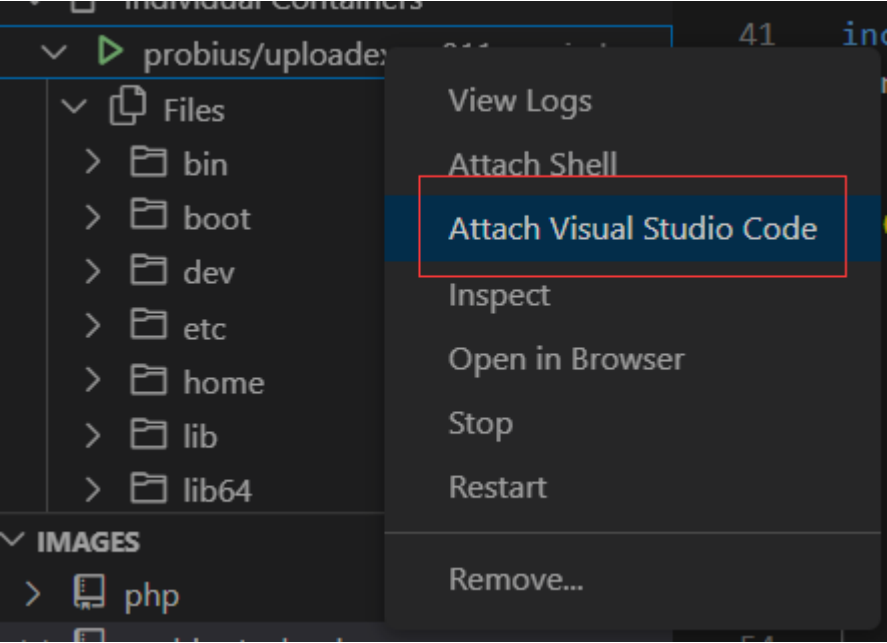


可以看到对于容器来说 修改是即时的（支持热更新的）

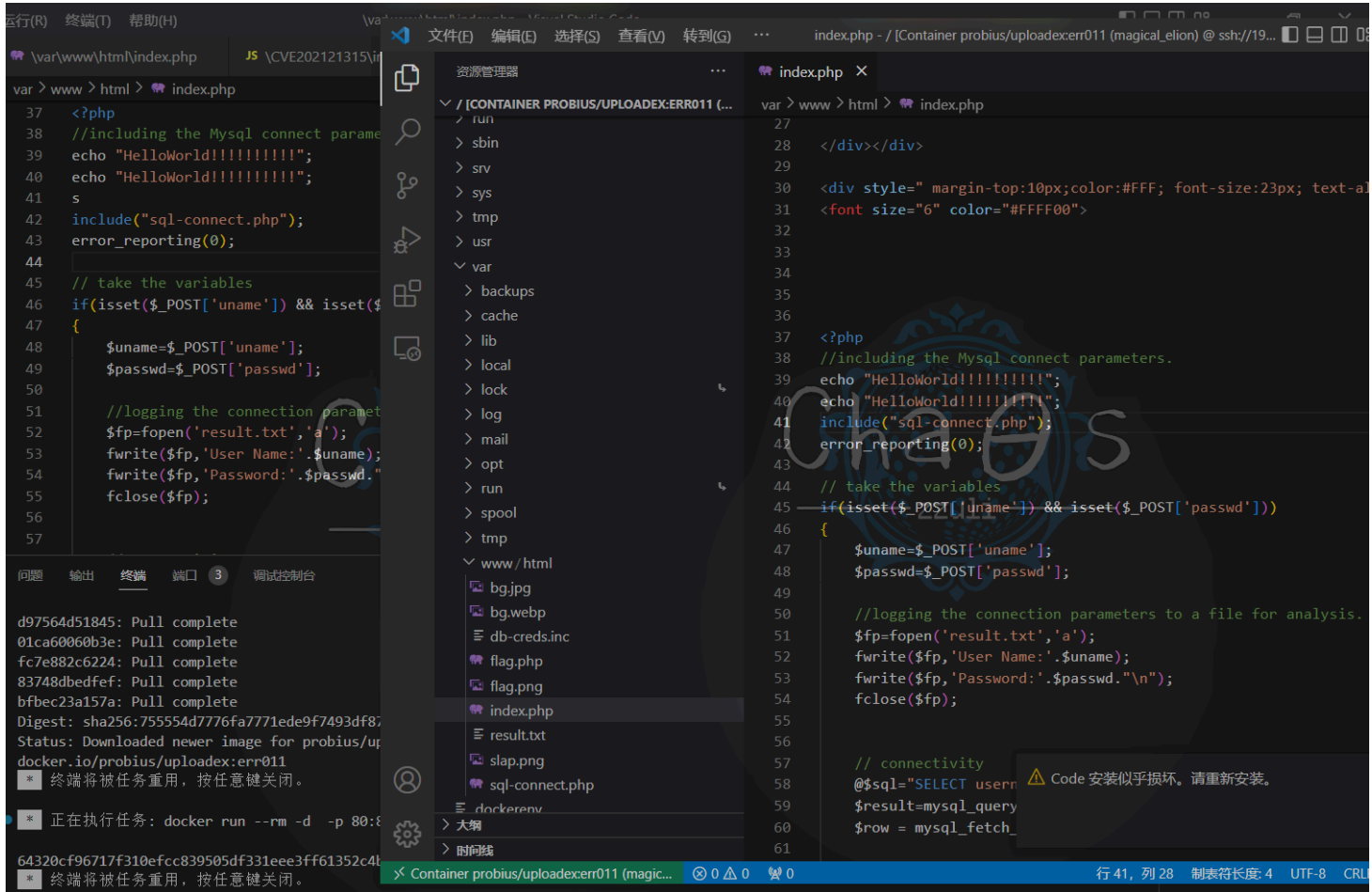


Attach Container (Advanced Edit?)

我们也可以用 VSCode 建立类似 SSH 的连接，直接连接到整个容器：



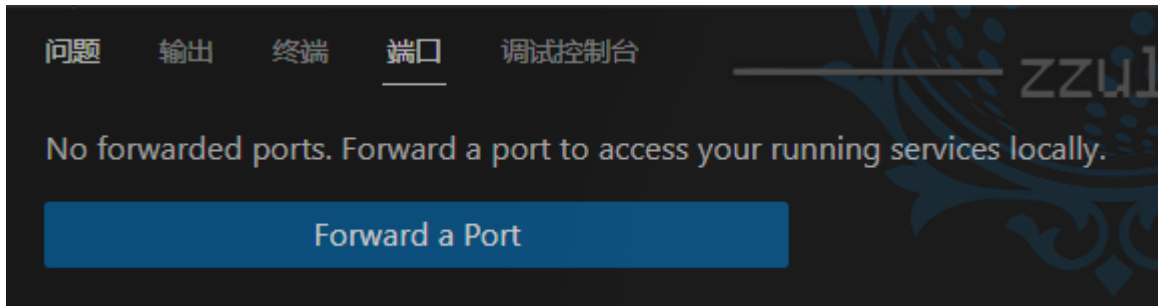
并且操作同步：



而且容器支持文件拖拽，相比单纯的 COPY 指令 拥有更好的体验（个人感觉

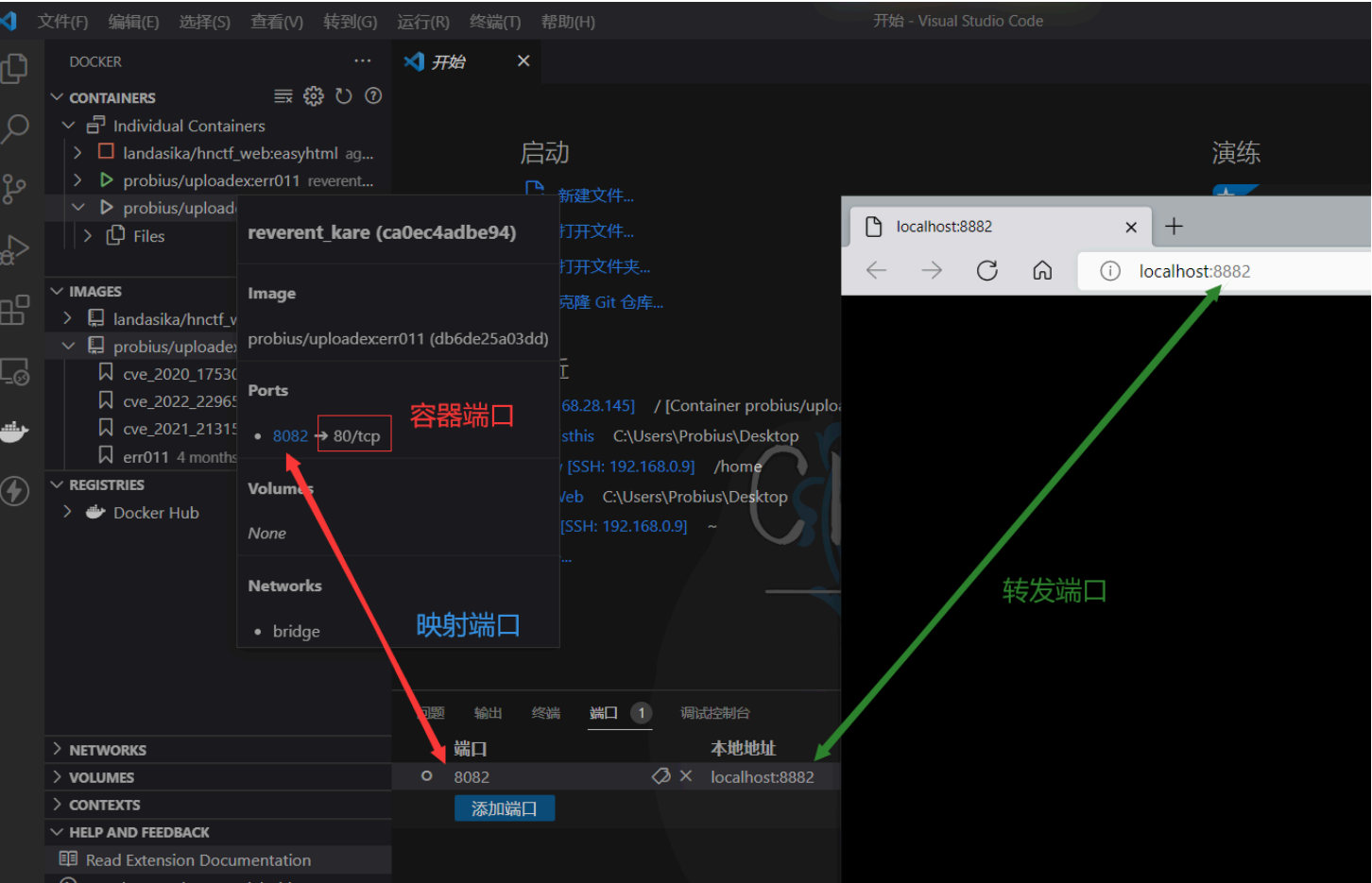
Port Forward

通过 SSH 直接将远程端口转发到本地，免除防火墙困扰 ww



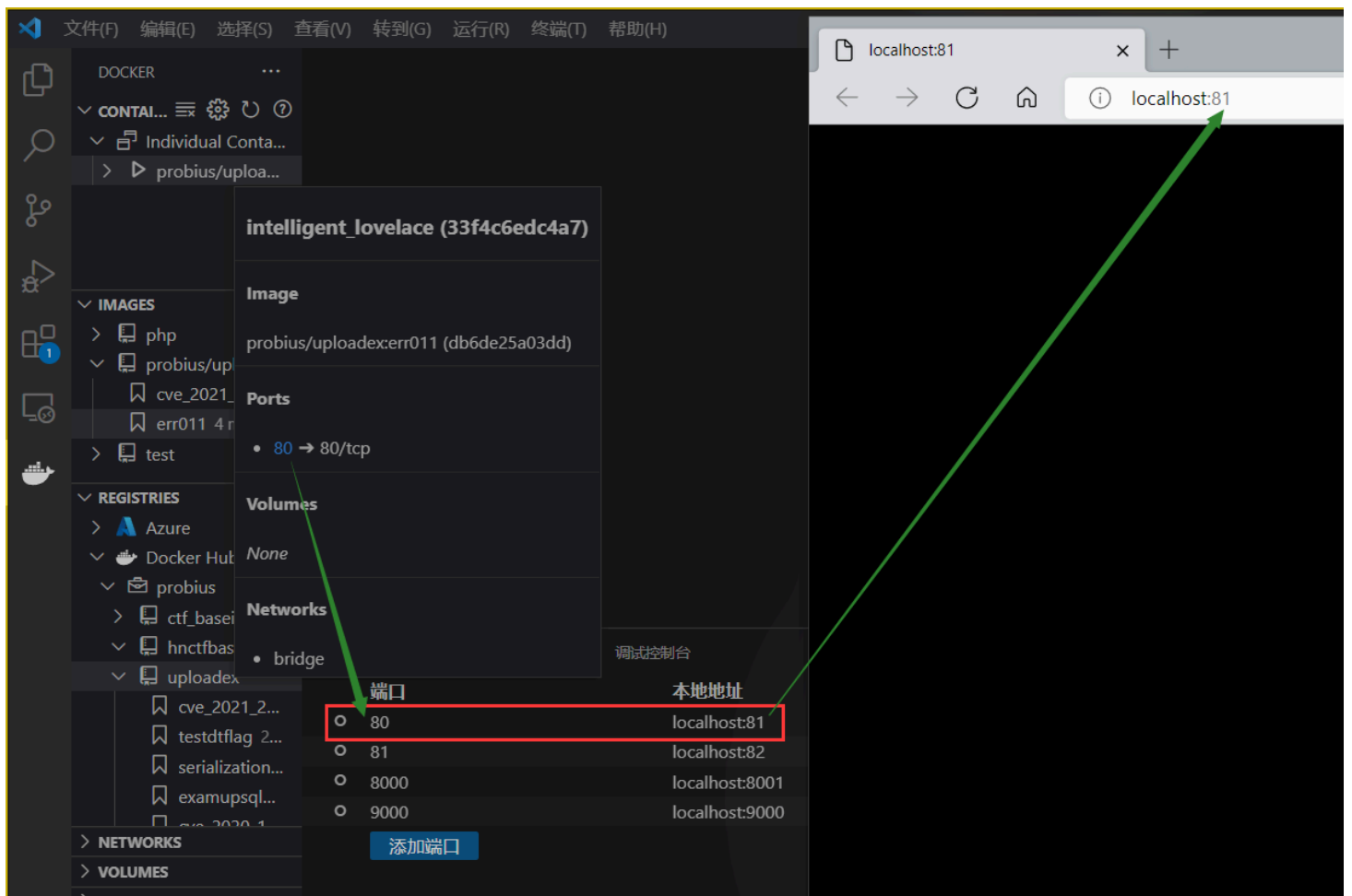
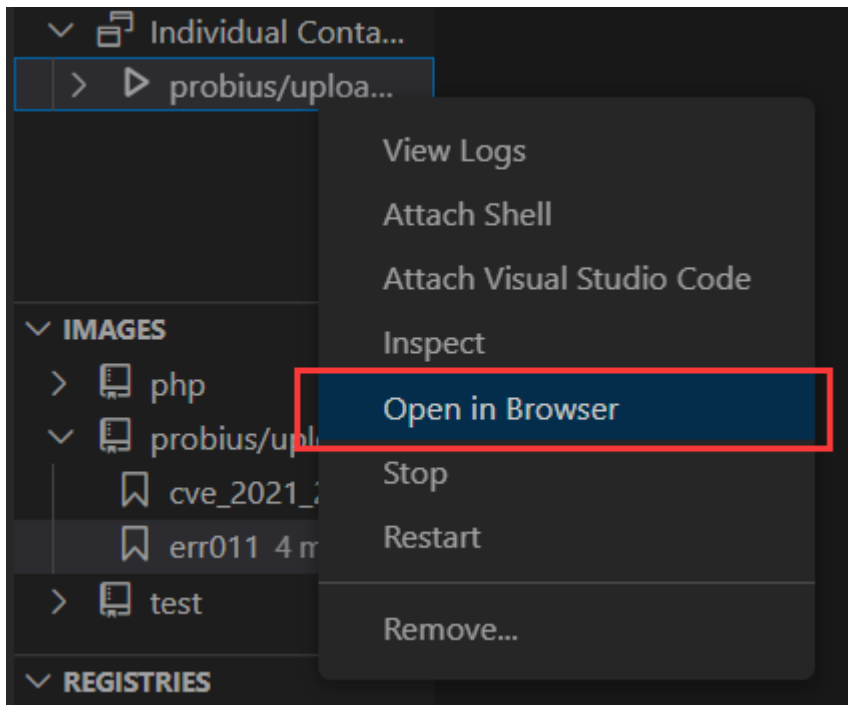
一般来讲在您启动容器的时候 VSCode 会自动配置 当然为了更好的区分和避免冲突 也可以手动分配

配置如图：



Open in Browser

如果您配置完成了端口转发 (或者 VSCode 自动为您完成配置) 那么点击 Open in Browser 将在您的默认浏览器中快速打开该端口对应的页面：



上一页

← 工具一览

下一页

Crypto 环境设置

→